

Bubble: An Interface for Programming in Three-Dimensional Virtual Reality Environments

Presented to
Brandeis University Department of Computer Science
Harry Mairson, Advisor

by
Aaron D Stone

May 2021

Contents

[1. Introduction](#)

[1.1 Bubble: Brief Overview](#)

[1.2 Analogous Systems](#)

[2. Context](#)

[2.1 Visual Programming History](#)

[3. Visual Programming Literature](#)

[3.1 EDGE \(Kleyn\)](#)

[3.1.1 Consistent Interactions](#)

[3.1.2 Knowledge Base](#)

[3.1.3 Event Relationships](#)

[3.1.4 Multithreading Visualization](#)

[3.1.5 Slots](#)

[3.1.6 Dialog Models](#)

[3.1.7 Transition Networks](#)

[3.1.8 Context Free Grammars](#)

[3.1.9 Control Flow Graphs](#)

[3.2 State Trees \(Rumbaugh\)](#)

[3.2.1 Local Variables](#)

[3.2.2 State Trees as Finite State Machines](#)

[4. Three Dimensional Visual Programming Research](#)

[4.1 Reasons for 3D Visual Programming](#)

[4.1.1 Visual Cortex](#)

[4.1.2 Screen Space](#)

[4.1.3 Space for Graph Structures shown visually](#)

[4.1.4 Expression, richer syntax, logic](#)

[4.1.5 Virtual Reality and Three Dimensions](#)

[4.1.6 Summary of Advantages](#)

[4.1.7 Modularity](#)

[4.2 Three Dimensional Visual Programming Research Examples](#)

[4.2.1 Holder Cubes, Pipes, Dataflow Metaphor](#)

[4.2.2 Visualizing Abstractions of Functions](#)

[5. Current Approaches](#)

[5.1 Multiple Interfaces in Blender](#)

[5.1.1 Blender Controls Issue](#)

[5.1.2 File Menus](#)

[5.1.3 Text Code System Problem](#)

6. Textual Programming

6.1 Reasons for Visual Programming for VR

6.1.1 Type Mismatch Bug in Text Code

6.1.2 Parameter order in traditional text code

6.1.3 Summary of Text Code Issues

6.2 Structural Capabilities of using 3D Objects as Syntax

6.2.1 Solving issues of text code in Bubble 3D Graphics System

6.2.2 Dynamic Update

6.2.3 Minimize Runtime Complexity

6.2.4 Type Safety using Visual Cues

6.2.5 Parameter Order Mismatch Solved with 3D interface

6.2.6 Misspelling parameters and variables

6.2.7 Unfilled parameters solved with 3D interface

6.2.8 Visual Syntax

7. Building in the Bubble System

7.1 The Bubble System

7.2 Bubble: Basic Visual Data Structure

7.2.1 Visual Programming

7.2.2 Visual Data Structures

7.2.3 Visual Namespace of Bubble objects

7.2.4 Slots: Visual Parameters of Bubble objects

7.3 Biblo Codebase

7.3.1 Bubble Inheritance

7.3.2 Modularity

7.4 Useful Terms for building more complex structures

7.5 Wave Graphs (Directed Graphs) for Visual Code

7.5.1 Wave Graph Example

7.5.2 A Directed Graph

7.5.3 Execution Order

7.6 Bubble Visual Syntax Rules

7.6.1 A Bubble with a Slot

7.6.2 Filling a Bubble into a Slot

7.6.3 A Slot with a Wave

7.7 Bubble with Slots and Waves

7.7.1 The 'Radius' Slot

7.7.2 The 'Click' Slot

7.7.3 The 'Hover' Slot

7.7.4 Directed Graph Form of the 'Sphere' Bubble

8. User Interface for Building 3D Graphics

- [8.1 Example: Data Visualization and Expression in Bubble](#)
- [8.2 Mockup of VR Application](#)
- [8.3 Stock Visualization Bubble](#)
- [8.4 Visualizing Abstractions with Visual Code](#)
- [9. User Interface of VR Application built in the Bubble 3D Graphics System](#)
 - [9.1. Feb 1-4 Diagram zoomed-out](#)
 - [9.2. Feb 3-4 Diagram zoomed in](#)
 - [9.3. Feb 4 Diagram](#)
 - [9.4 Feb 4 Detailed](#)
 - [9.5 Feb 4 2am-5am](#)
 - [9.6 Feb 4 2am](#)
 - [9.7 2am Tweets](#)
 - [9.8 Tweet: Detailed](#)
 - [9.9 Zoom Out to Feb 4](#)
- [10. Discussion](#)
 - [10.1 EDGE, CUBE and Bubble Comparison](#)
 - [10.1.1 Knowledge Base](#)
 - [10.1.2 Dataflow and variable setting similarity](#)
 - [10.1.3 Transition Networks and Waves Comparison](#)
 - [10.1.4 Context Free Grammars, Interactive Pushdown Automata & Wave Graphs](#)
 - [10.1.5 Window Occlusion Problem \(EDGE\)](#)
 - [10.1.6 Three-Dimensional Solution \(Bubble\)](#)
 - [10.2 Bubble Compared to Rumbaugh's State Trees](#)
 - [10.2.1 State Trees](#)
 - [10.2.2 Bubble](#)
 - [10.3 Discussion on Weaknesses of Visual Programming](#)
 - [10.3.1 Bubble](#)
 - [10.3.2 Screen Space](#)
 - [10.3.3 Types in Bubble](#)
 - [10.3.4 Higher Order Functions](#)
 - [10.4. Future Direction - Type Graph for Visualizing Bubble Edits](#)
- [11. Summary and Conclusion](#)
- [12. List of Figures](#)
- [13. References](#)

Acknowledgements

There are a few people who I would like to thank for their help with research and support during the process of writing this paper.

Mark Russell has been involved in the programming building process of the Bubble System and theory for the system. His insights have also been helpful for writing this paper on the visual programming industry. See his thesis for more information on the system and its structure from a language perspective.

Harry Mairson has been an incredibly helpful advisor and person to talk to about the ideas and theory of the system, especially for helping explain ideas with more clarity and giving a realistic outsider perspective on the system, in addition to making sure that the goals are realistic. He was a driving force behind making sure that the paper would be written to accommodate a layman's understanding, and that considerable literature would be reviewed for context. Additionally, his work on the Digital Amati project has been a great inspiration.

Timothy Hickey has given valuable perspective on the system and context in the 3D graphics industry and the state of the art of 3D Graphics. His insight on Blender and related systems has been incredibly helpful.

1. Introduction

Virtual Reality -- the use of graphics projected into three dimensional space viewed by users with a headset -- has become a large part of the gaming market in recent years. Many companies focusing on secretive research in the space have been unsuccessful as of yet. Currently, most code written for Virtual Reality (VR) systems and applications is written using a textual interface for programming. This is the common way of programming which, of course, yields many benefits in the ubiquity and the reliability of text for representing structure. Between approximately 1980 to 2000, some research papers focused on creating visual programming languages and systems for building interfaces using visual representations rather than text to build structures. Currently few systems exist that use three dimensional (3D) visual programming for VR systems.

Three dimensional visual programming systems utilize an extra dimension of space, showing data *lazily*, meaning having a generalized graphical representation which can be clicked on or zoomed on to view the data with more density. This added dimension (depth) and graphical component makes the data easier to traverse by zooming in and out of and clicking on visual three dimensional objects. Since not every level of generalization must be shown at once, three dimensional (3D) visual programming greatly facilitates building and using 3D software.

The goals of this thesis are:

1. To outline history and current approaches for visual and non-visual programming structures for use in three dimensional environments.
2. To introduce the Bubble 3D Graphics System, presenting a mockup of part of the process for building VR applications using the Bubble system
3. To discuss the Bubble System in the context of related research in the subfield of visual programming

We introduce the Bubble 3D Graphics system presenting a mockup of the process for building VR applications using the Bubble system. and a mockup of a VR application, a stock graph application utilizing dynamic cross data visualization, where different datasets can be imported simply by filling a slot, using data from the codebase.

1.1 Bubble: Brief Overview

For programming and graphical building capability, Bubble is meant to be an easier way to build structures which can be visualized in three dimensions and can:

- Build VR applications and drag objects from a visual codebase into three dimensional scenes, add visual objects to 3D scenes

- Import Objects with symbolic visual representations; 3D interface used to edit Objects and their variables

- Use a visual directed graph to *visualize the dataflow* of the code structure (visual representations of code).

Developers familiar with Blender might think of the programming interface in Bubble as a ‘logic bricks’ coding interface except with three dimensional blocks, similar to Scratch, with dragging and dropping to change the values of variables. However, whereas Scratch is mostly used for learning to code, Bubble is a system for programming complex Virtual Reality applications, and is designed to allow for a large amount of expression through graph structures shown visually, branching in all directions in 3D space. Three dimensional space provides more space for displaying directed graphs of the structure of the applications being built. The Bubble system visualizes the dataflow with code objects or building blocks, called ‘bubbles.’ A **bubble** (class/object) is a visual representation of a class/object code, which the user manipulates within the 3D user interface. Many graphics programmers who currently use other 3D graphics software might benefit from Bubble to create applications, since it requires less of a learning curve, with fewer specifically curated tools for building interactive VR apps.

In the long term, Bubble seeks to become a hub of VR applications. Our goal is for users to readily recognize Bubble VR apps based on its distinctive, consistent design, all built with the same graphics tools from the Bubble System. Users will be able to search through a variety of apps in the Bubble VR App store, which they can find and download to their devices (VR headsets, computers).

1.2 Analogous Systems

NeosVR is similar to the Bubble System, and uses a graph structure similar to Blender logic bricks (see Chapter 5) instead of text code, for the most part. It is used

for building three dimensional graphics, and incorporates programming capabilities into virtual reality. The programmer can see the change in the graphics live as the structure of the code blocks have changed. Bubble is different from neosVR in that the whole interface for building structures in Bubble are three dimensional blocks, whereas 'neosVR' is two dimensional panels in a VR scene, used to build structures by connecting wires to these two dimensional panels, which are code blocks [1].

In recent research, Harkonen describes how Entity Component System model (ECS), a Data-Oriented Design approach is used instead of the traditional object oriented design model [2]. Harkonen describes the history of ECS, and how it began with game development with "Thief: The Dark Project" in 1998, where components were reused to build the game, and the same components were used for building a second game [2]. Bubble does not use Data Oriented Design, unlike ECS, but it does use a visual interface for visualizing the graph structure of programs, which is similar to ECS. The Bubble system adheres to Object Oriented principles for its structure. The object oriented approach and visual programming combined enables for a visual interface for manipulating Objects, 'bubbles', and building and editing these 'bubble' Objects. This makes it easier to build complex structures because each part of the structure can be visualized as a bubble object.

Raffaillac and Huot describe how ECS can be used for input handling, and building graphical user interfaces. All of the objects in the program are entities, which store ID's, while components are properties of entities like 'color'. Their paper on game development using ECS outlines the difficulties of using conventional methods for input handling and the many systems and tools required, making it very challenging even with today's technology, which makes a system like ECS useful for building games [3].

2. Context

Visual Programming research flourished from 1980-2000 and has been somewhat forgotten since the ubiquity of the Internet and Window interfaces, which are built using conventional text code. Because of the desire to build applications and push features incredibly quickly in the Internet age and smartphone application development surge, companies never stepped back to think about using a visual programming approach because it requires additional overhead and effort in the short term. Visual programming is building both 'front-end' and 'back-end' structures using visual representations to visualize the structure being built. the

possibilities for innovations and contributions of visual programming systems and research to the future of technology are nonetheless profound.

Combined with today's technological and computational leaps, the potential of visual programming is as powerful as ever. This is especially true since Virtual Reality (VR) is more realistic now than ever before, due to visual programming techniques. VR is the ideal interface for visual programming, and the two ideally complement one another. Najork mentions in "Programming in Three Dimensions" that his visual programming language, CUBE, was intended for VR [4]. He writes that his plan for the future of the programming system was to be used in VR with only 3D interfaces, and 2D interfaces (or file menus). Now that VR is more realistic, especially with the help of visual programming, we are entering an era of both Virtual Reality and Visual Programming.

2.1 Visual Programming History

Sketchpad was the first graphical user interface created [5]. In 1966, William Sutherland implemented the first visual language [6], which utilized visual dataflow, and picking an operator using a menu could connect operators using a light pen, and set variables to circuits, which the programmer could watch the real time execution [6].

Sketchpad sought to make visual manipulation and visual abstraction available to users and programmers. Features of Sketchpad, which were novel at the time [5] include:

- 1) Drag and drop
- 2) Click to select an Object
- 3) Zoom
- 4) Inheritance
- 5) Objects and instances, with visual representations

Pygmalion, an initial visual programming system influenced by Sketchpad, first used the term 'icon' showing the power of an image representing abstractions. Sketchpad influenced the Xerox Star user interface and applications, inspiring 'drag and drop' and clicking on visual objects, and also inspired the Windows and Macintosh interfaces [5].

Though there is not yet a commonly used system supporting visual manipulation and visual interface for abstraction, programming using representations or icons might make it easier to build abstract structures [5]. Sketchpad influenced Simula, an initial object oriented language, but there is no system like Sketchpad today for building object oriented structures using visual representations of abstractions and dataflow; text code is the only everyday option [5]. Simula, an initial object oriented language, and Sketchpad influenced Alan Kay's Smalltalk [7], which was developed at Xerox, and was one of the first widely used object oriented languages.

Kay, a pioneer of object-oriented design, compares visualizing objects to cells in biology, which send data to other cells [7], a concrete visual representation of the data passing abstraction. This idea is similar to the Bubble System, outlined later in this paper, where we describe visualization of dataflow code objects ("bubbles") and data passing between bubbles can be constructed through a bubble filling the slot of another bubble. Waves (code), attached to slots then fire when a slot is filled or a value is assigned. Visualizing dataflow is simplified in Bubble, since waves are shown in order topologically, one after another, in the format of a directed graph, where one wave points to the next wave in order of execution.

Kay observes that open source software takes up hundreds of millions of lines of code and that understandable and concise language design would be useful, and could be written with far fewer lines [8]. In this thesis, we outline a concept of using visual representations of objects, 'bubbles' to build structures with enhanced modularity, dynamics, easily visualized inheritance, and a structure which maximizes understanding of large systems for novices while enhancing the expressive capabilities for experts.

3. Visual Programming Literature

Below, we outline important examples of programming systems which are useful for analyzing Bubble and its context in the history of both systems for building 3D graphics and visual programming capability: EDGE (Kleyn) [9], State Trees (Rumbaugh) [10], CUBE (Najork) [4]. These works all describe systems for visualizing interfaces and the control flow of programs, to change the structure of the program.

The papers below are helpful for understanding the industry context: visual programming systems. An understanding of the Bubble System is not complete

without understanding the history of visual programming and the unfortunate lack of influence of visual programming system research from approximately 1980-2000s, due to a lack of computational power in past eras, which made it difficult to support high quality graphical representations of code. Any visual programming system had too much overhead and were not able to support complex graphical systems running smoothly.

3.1 EDGE (Kleyn)

Michael Kleyn and Indranil Chakravarty describe a graph system for representing user interfaces and the control flow of applications. This graph visual representation is helpful for visualizing the structure of programs and building applications [9].

3.1.1 Consistent Interactions

Kleyn mentions that the reason for building a design tool and directed graph is not only to visualize interaction dialogs for modularity but for dialog consistency, meaning that the **interaction paths to execute a task are consistent across applications** [9]. This is helpful for both programmers and users because having consistent patterns of input paths means that using the application becomes automatic, and code for the application can be reused because of the consistent control flow.

This is more difficult with multithreaded dialogs with 2D windows, because there are multiple windows required [9]. Multiple windows are not required in three dimensions, since the graph and the codebase can exist in the same environment, and there can also be multiple interaction processes occurring simultaneously in a single 3D VR scene. Three dimensional environments can have sensors handled as multithreaded interactions. An example is two hands doing gestures at the same time. However, with a keyboard and mouse, it is almost impossible for the user to interact using multiple input types at the same time (i.e. typing and scrolling at the same time).

Further, handling multiple simultaneous interactions is impossible with 2D because most 2D interfaces have a different interface and different system built for every use case -- a different file menu structure with different tools for each use case.

Bubble has the ability to visually represent the control flow of 3D interactions, representing 3D scenes as nodes using a ‘scene graph’ format, which is a directed graph of all of the interaction paths which the user can take in the system.

3.1.2 Knowledge Base

In EDGE], developed by Kleyn, ‘Events’ are stored in a knowledge base, which are leaves of the EDG graph [9]. The EDG graph represents multithreaded user input paths. This is similar to the Bubble System Wave Graph and a codebase system described later in this paper. In the discussion section the similarities between the two systems are analyzed.

3.1.3 Event Relationships

Relationships between events and states are stored in the knowledge-base in EDGE. Events in the system are interface changes that happen in response to user input, while states are the program state or interface state which change in response to events. The knowledge base is used along with the event graph in the EDGE system. The user can visualize timelines of events, and events caused by actions in the graph [9]. State variables represent the state of the interface at a given abstraction level, which is propagated by events. Multithreading is visualized in the system using the graph [9].

3.1.4 Multithreading Visualization

Visualizing multithreading in EDGE using the EDG graph is similar to the Wave Graph in the Bubble system, except wave graphs branch in all directions in three dimensional space. In both, the programmer has the ability to visualize the branches of control flow represented by going in different directions graphically.

3.1.5 Slots

In EDGE, the programmer selects and changes the values for slots, which are variables, using a visual interface. The user uses a dropdown menu to enter the new value. Kleyn mentions that there is the inherent issue of the window hiding the slot value which is being updated. This way of using file menus to change slot values is more difficult than the same functionality in Bubble. The EDGE file menu structure is difficult to traverse because it requires selecting a command in the commands menu [9], which is attached to an editor window, in addition to typing in the correct value. This is a tedious process in a 2D interface, and can become difficult to select all of the proper settings without accidentally clicking the wrong button (see the discussion section on how this problem is solved in Bubble.)

3.1.6 Dialog Models

Kleyn describes how ‘dialog models’ are used to show the states of a program which contain interactive capabilities. He mentions three types of these models: Transition Networks Context Free Grammars, , and Event-Based [9].

3.1.7 Transition Networks

The transition network is a visual representation of state transitions in a program. The edges of the graph can represent functions which activate when a state transition occurs. Each vertex of the graph can then also be a sub-diagram, which is then recursive. This is quite an elegant approach and visualization for representing the states of a program. Kleyn does mention that this system creates a large amount of states and can be tedious to use for larger implementations [9].

3.1.8 Context Free Grammars

Kleyn describes Context Free Grammars in the setting of user interface visualization as graphs with the syntax necessary to describe dialog-based systems formally [9]. These graphs define when a user can execute given actions. Filtering actions allowed in the system is executed by a parser. Kleyn mentions Interactive Pushdown Automata, which are parsers that are used as an interface visualization system for state changes to the interface of the program [9].

3.1.9 Control Flow Graphs

Control flow graphs, which are described by Kleyn in EDGE, come from previous research on using a visual graph to represent a system in an object oriented manner [9]. Using control flow graphs to represent the structure of the interaction paths of programs is designed to make it easier to create new interfaces by combining objects. These interfaces are called ‘editors’ which are domain specific. An ‘editor’ is a window with ‘interaction commands’ based on the context. These graph views allow designers to control what is shown in the interface and its layout. Designers can use these editors to change and visualize interface, while using the EDG graph to visualize and change the control flow of the program and visualize events and actions.

Kleyn was one of the first researchers to use directed graphs to represent the interfaces of programs, and studying his research is important for understanding the future potential of using directed graphs for visualizing the interface and underlying structure of programs [9].

3.2 State Trees (Rumbaugh)

Rumbaugh describes a visual representation system based on a directed graph. ‘Actions’ are commands, and can happen in response to events [10]. State trees are asynchronous and run externally, run from the interface, catching and interpreting events and then running the actions. Its ‘externally driven’ design differs from internally driven, synchronous systems. The state tree is able to represent different potential branches of the interface, representing a multithreaded system rather than a single thread of interface actions [10]. Local variables are attached to nodes of the state tree, storing information on the state.

3.2.1 Local Variables

Local variable scope is within the subtree of the state, and can be accessed by the states of the subtree. An action can change or access a variable or use it to pass information to a different area of the tree. The example is given of a ‘polygon’ state which would contain local variables: the number of vertices and the list of vertices. Local variables store state and sub-state information, similar to how a slot is a variable, which stores a value in the Bubble System. See Discussion for a more detailed description.

3.2.2 State Trees as Finite State Machines

In outlining a process for making finite state machines from state trees, Rumbaugh mentions event traps using inheritance to make the handling of user actions easier. Interface event traps are conditions of the interface to implement input handling using the graph. State trees change the interface of the application itself. The programmer can edit the tables to change the interface instead of rewriting code to change the interface. These graphs are designed for easing the visualization of the control flow of the interface in addition to making it easier to change the interface without writing code, improving modularity [10].

4. Three Dimensional Visual Programming Research

Najork’s CUBE is an example of the first visual programming system where logic and 3D Graphics are combined [4]. Marc Najork’s research provides important industry context on visual programming. CUBE was the first visual programming source found in this research, and allowed for the exploration of the past research in visual programming. Najork references Ephraim Glinert, who was one of the

earliest to advocate 3D programming [4], suggesting that the field of computer science should aim for ‘revolutionary rather than evolutionary means of programming’ adding that the implementation to do such will be ready when necessary [11].

4.1 Reasons for 3D Visual Programming

Najork effectively outlines the most important reasons for visual programming [4]. Below we summarize his research and commentary on visual programming.

4.1.1 Visual Cortex

The human brain is set up for visual processing, since the visual cortex is faster at processing than other areas of the brain, and allows for random access at different parts of the picture being viewed. Najork mentions that visual syntax for programming allows for rich language [4]. In addition, text is one dimensional, and visual representation can convey more information [4].

4.1.2 Screen Space

One drawback mentioned against visual languages is screen space. Najork refutes that claim, writing that systems such as XEROX PARC Information Visualizer has abstract structures like trees, graphs, tables shown in 3D [4]. He notes that using 3D actually makes for more efficient use of space, since the dimension of depth is available for more space to build three dimensional structures. He also states that rotating the scene creates more opportunity for deriving information [4].

4.1.3 Space for Graph Structures shown visually

Graphs are easily displayed in three dimensions, since graphs can branch using the dimension of depth so that the edges do not overlap, and nodes do not crowd the screen. Therefore, graphs are easier to understand when branching in three dimensions. Visual languages use the dataflow metaphor, displayed as a graph in 3D [4]. Najork mentions a study from Ware and Franck finding that the brain can visualize triple the amount of information using a graph in 3D compared to a 2D graph [12].

4.1.4 Expression, ‘richer’ syntax, logic

Raeder (one of the earliest scholars on visual programming) describes the advantages of using the visual cortex, lending credence to the idea of visual languages [13]. He describes text as a single dimensional stream of words. However, pictures have multiple dimensions for information, in addition to being able to show

visual characteristics like shape and color [13]. Thus, he concludes that visual languages have a more expressive syntax than textual languages.

4.1.5 Virtual Reality and Three Dimensions

Najork mentions that a visual programming system works perfectly in a Virtual Reality environment [4]. Visual Programming using direct manipulation capability for objects would be possible with 3D graphics and sensors or handheld controllers with sensors, where the user would be able to manipulate objects using their hands, instead of using a mouse. This solves the screen space issue, since the perspective would be the head of the user, and moving the head would move the part of the environment being viewed, which is more intuitive than using a mouse. Najork suggests that a VR programming environment could make it easier to develop VR programs [4]. A software called *Lingua Graphica* aims to do this exactly and was developed by Lockheed Martin AI Center [14]. It was created because building VR software in a VR system makes it easier to “edit-compile-debug” because it would all be in one VR environment, rather than developing VR software using text code and then checking and testing the software in VR. Building VR applications in a VR environment makes for a more efficient programming process [4].

4.1.6 Summary of Advantages

In summary, Najork’s arguments on the advantages of a visual programming system [4] include

- 1) The human brain is built for visual processing.
- 2) Random access is possible through visual processing rather than sequential access through text. A viewer can gain general or specific views of a picture, since they can simply change their viewing position to different parts of the picture and also change the generality.
- 3) A human brain processes images quicker than text, in near-real time.
- 4) Programmers already express algorithms and data structures using visual representations of structure.
- 5) Visual representations of code allow for more efficient, non sequential and non-linear processing of structure as opposed to text.
- 6) Visual language allows for conveying information with less syntax, since there can be visual representations for a wide variety of objects, similar to character-based languages.
- 7) Visual representations and pictures can convey abstract concepts.

4.1.7 Modularity

It should be noted that building structures using visual representations are a modular way of building, since the creator can reuse code through referencing visual representations of objects. The structure can be more concise, since its components itself are modular and can be altered in a simple way.

4.2 Three Dimensional Visual Programming Research Examples

Najork mentions some important examples of the advantages of programming visually, which have been summarized below [4].

4.2.1 Holder Cubes, Pipes, Dataflow Metaphor

Najork describes the dataflow metaphor and a programming system using ‘holder cubes’ which contain terms that are also cubes, and are equivalent to variables in text languages. Holder cubes connect to each other via a ‘pipe.’ He describes that if there is a value inside of an empty holder cube, the empty cube is filled with the value that is put inside of it [4].

4.2.2 Visualizing Abstractions of Functions

Najork notes the ability to represent higher order functions (a function accepting functions as parameters, or a function which returns a function) with a visual, or disambiguate its contents by clicking or zooming in on the function to see the functions which it takes in. Higher order functions allow for *code reuse*. Visual programming systems are perfect for visualization of abstractions where higher order functions are being used, a key advantage of a visual programming system [4].

The above literature review has illustrated the history of the field of visual programming, the advantages and disadvantages of the approach, and different research projects; CUBE, EDGE, and State Trees, which exemplify the potential for using visual representations for building structures, especially using directed graphs.

5. Current Approaches

Game developers currently use Blender, Unity or the Unreal Engine for building 3D graphics for gaming and VR applications. Below, we discuss the 3D graphics systems space, and how it relates to the Bubble System.

Hickey describes an ‘Introduction to Programming’ course on Blender. He mentions the logic bricks, which are used in Blender to set up inputs and set variables for games [15]. ECS (described above) is a visual programming system which is used for logic in Blender. Below we describe Blender and how Bubble may be able to fit into the 3D Graphics software space.

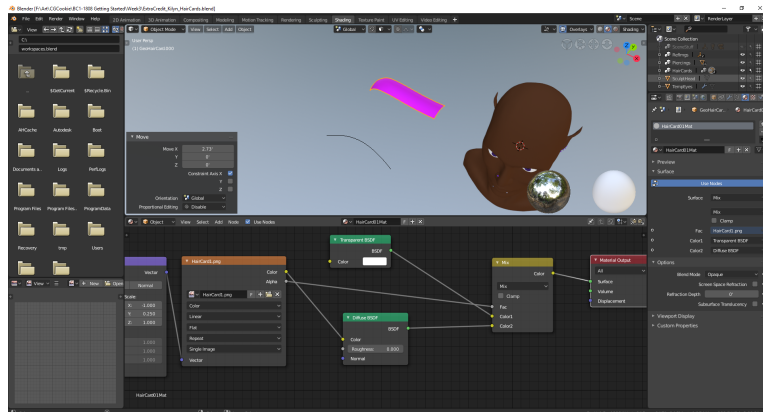
5.1 Multiple Interfaces in Blender

It can be difficult for developers to use the Blender system, since there are multiple disconnected interfaces:

1. Many different file menus, each with a different interface. See left and right sides of Figure 1.
2. 3D visual output. See the top middle of Figure 1.
3. Node editor. See the bottom middle of Figure 1

The user is forced to visualize and understand how these interfaces connect, making building applications in the system difficult.

Figure 1: Multiple Interfaces in Blender



5.1.1 Blender Controls Issue

With Blender, zooming in and out and panning in the 3D interface works differently than other systems, and requires mastery because there are more inputs. Coding in the VR scene itself is easier, since there are more inputs available, through the use of hands, in addition to gestures being performed on a 3D object.

The drop-down tabs and menus on the sides can be difficult to navigate, since there are many possible paths and items to click on. There are many different possible settings and modes in the interface that many users are challenged with using.

Zooming, panning, and grabbing and dragging objects in Blender can be difficult to master.

5.1.2 File Menus

In Blender, the user is forced to traverse nested file menus to find tools, objects, and settings.. There can be a steep learning curve for each new interface made for each file menu use case. Further, there is not much space in 2D file menus, which become similar to a game of ‘operator.’ File menus need to be squished with the interface for visual output, and they are often on top of one another. These interfaces are discontinuous. Finally, the user is forced to learn many different key combinations. The current solution for this is macros, which are not user friendly, and require extensive memorization.

5.1.3 Text Code System Problem

In Blender, text code is discontinuous, with text files in many folders. Also: text and visual output are separate. Users must line up separate systems manually, internalized, without an interface for visualizing. Users must visualize the structure of code and graphics, which are in disconnected interfaces. There is no easy way to know the file structure of the textual code. Further, the programmer is forced to know how the code interacts with Blender.

6. Textual Programming

Some inherent advantages of text code and limitations of 3D Graphics systems for building structure need to be acknowledged. Mairson outlines some advantages of text code, where the builder is able to see precisely what is being built, rather than being confused by the equation representing a shape [16]. He describes a desire for language-based expression through a textual interface rather than WYSIWYG (“What You See is What You Get”), to retain the exact text of what is being built, verbatim. In Mairson’s case, it is about being able to see the text which describes the math and code underlying the geometries being built. The need for a textually based abstraction rather than visual indirection is something which textual languages have not fully taken advantage of, but using text for abstraction is an idea which Mairson’s Digital Amati has used for expressive capability. The Bubble System seeks to do the same form of abstraction from equations, while incorporating visual indirection to represent code with visual representations.

Bubble also attempts to use abstraction for expressive power, similar to Mairson's 'Digital Amati.' Instead of the syntax of "circle()" to create a circle, Bubble uses a visual representation drawn from the codebase. However, the base idea is similar in that both systems are directing from abstractions using a simpler representation. Mairson shows that optimizing the code for indirection and ease of use so that equations are not needed for dealing with objects, which makes the building experience easier and more genuine than other programming languages. Thus, the Digital Amati language has greater ease of use compared to other languages, and programmers are able to simulate the experience of using a protractor to create a circle, instead of using equations [16].. It indirects upon the equation using the name of the object, which is a perfect example of utilizing the power of interface. Interfacing for ease of use is something that the Bubble System attempts to do, which will be described later in this paper.

Mairson aptly shows the inherent purity and potential for interfacing, which is not available using a graphics system or WYSIWIG for building. What the WYSIWIG gains from abstracting above text code it loses, since the text code is lost and has to be viewed separately if the programmer wants to see the code in its textual format.

6.1 Reasons for Visual Programming for VR

Listed below are some of the most common issues with text code that can be prevented using a visual 3D syntax, which the Bubble System uses to minimize bugs.

6.1.1 Type Mismatch Bug in Text Code

In text code, sometimes a programmer fills a parameter of a function or method with the wrong type. The name seems correct, but there is often a type mismatch of the type of what you are filling the parameter with and the value that the programmer is putting in to fill the parameter.

For ex. Sphere class:

has "changeColor()" method which takes in one parameter:

You change the color by typing
`changeColor(red);`

If a programmer types `changeColor(5);` that's an issue, but the programmer might not know that until after hours of bug fixing. If a programmer types `changeColor(orange)` but `changeColor` can only accept let's say, objects of type "wrappedColor" for example, and orange is of type "color" there is an issue.

6.1.2 Parameter order in traditional text code

Let's say we have a Square class, with method, setProperties(length, color), which has a length parameter and a color parameter. But since parameter order matters in text code, we must write code to fill that method and update the sphere like this:

```
Square s = new Square(5, red);
```

Shown above is the strict way of writing text code which requires set parameter order. Of course, it can only take in parameters of type length and object. Let's say we have the following code:

```
Square s = new Square(red, 5);
```

The above code would cause an issue which may take some time to solve, especially with a class in a text file with hundreds of lines and logical and structural issues. Sometimes the code will not run, and the programmer doesn't know why, or the code will run, even if it has serious structural issues and bugs. Both are difficult to solve, because there is no way of knowing easily for sure the issue without debugging. Sometimes, the programmer has to rewrite all of the code to fix the issue, because it is so minute and subtle, that they must simply give up trying to find it. In Bubble, the programmer fills parameters using the visual representation and name of the parameter, rather than requiring strict parameter order.

6.1.3 Summary of Text Code Issues

Parameters: Forgetting to pass in a parameter for a method, can create bugs which are hard to fix. Filling parameters for a method with an object of wrong type by accident can also create bugs which are hard to solve.

Bugs: Programmers may not realize bugs until after the program has been run, which sometimes only happens after a large amount of code has been written. It would be helpful to have a graph of the program, so that structural bugs can be caught before they are created, and parts of the code that contain logical holes or gaps can be found visually and immediately, rather than after the code has already been written and run.

Additionally, error messages refer to code and the cause of a problem, using disconnected interfaces, as well as only reflecting a shallow trace of the call stack, doesn't reflect any knowledge of data dependence or the structure of the actual program being built.

6.2 Structural Capabilities of using 3D Objects as Syntax in the Bubble System

6.2.1 Solving issues of text code in Bubble 3D Graphics System

The issue of parameter order mentioned above is solved for the user in Bubble, because just like Excel's feature of automatically updating the cells when an equation is typed in or when the equation is changed, in Bubble, the graphical output is shown immediately when a slot is filled.

6.2.2 Dynamic Update

In the Bubble System, the programmer sees the output at all times while coding, since output updates when a slot (parameter) is filled. Since the programmer sees the output of the code immediately, the system minimizes bugs because of the ability to catch logical issues which are shown in the output while the code is being updated. This minimizes 'blind coding', since all structural changes are checked and 'run' immediately. Since the visual code is being viewed along with the visual output of the program, each dynamic update on a slot fill has no discontinuity.

For example, given a bubble object "Sphere" with a "color" slot which is filled with the "black" bubble, the color of the "sphere" automatically updates to becoming 'black,' the bubble which was dragged into the slot. The output dynamic updating is a visual way of showing type mismatches to reduce bugs and make the visual graphics system fail safe (since the programmer sees the wrong output immediately if a wrong type is passed into a parameter.) This is a huge contrast to normal code, where you do not know for sure if a parameter being filled/passed in has been done correctly. To check for this in normal code, the programmer usually has to print to the console, or run the entire program to check for bugs. This is more difficult when there are parts of the program which have not been written yet, using the conventional text coding approach. In the Bubble System, one of the main ideas is using code which is generic and already interfaces perfectly with other components of the system. There is less blind coding involved in the Bubble System, since there is no risk of a failure to make a leap in changing the code structure because all of the components are pre-interfaced.

6.2.3 Minimize Runtime Complexity

Since the Bubble System's types are constantly being checked via Dynamic Update, (every time a slot is filled) type checking at runtime only needs to be done minimally. The Type Graph adjusts itself every time that Dynamic Update (the filling of a slot) happens, similar to the structure of an AVL Tree, which always

updates itself after every ‘add’ to its structure. Additionally, the system automatically keeps track of dependencies between types, simplifying and checking in a graph format. The non-linear structure of type checking minimizes runtime by ensuring that succession of instructions/waves signifies absolute dependence... making ‘the succession of lines of code’ an indicator of more than just succession (the extra meaning that this imbues ‘then’s with is what the Type Updating algorithm depends on for its speed. Thus one might say that ‘runtime’ is constantly occurring via Dynamic Update (since every Dynamic Update runs the code, thus become a granularization of runtime) and the graph of the full Type System is constantly updating itself, and each change is minimal because only what must be updated on the graph (new changes) are being checked, rather than the whole structure. The runtime complexity of Dynamic Update can only be of the order of the size of change(s) being made, rather than on the order of the whole graph (ie, if the user specifies X number of changes, the complexity of the dynamic Type Graph restructuring is $O(X)$.) Thus the system is never slowed down by type checking at runtime, since it updates constantly, but only changes what is absolutely necessary in the Type Graph similar to how an AVL Tree will update itself every time it adds to itself. Hence the runtime complexity of code is unaffected by any overhead of the Dynamic Update algorithm.

6.2.4 Type Safety using Visual Cues

The Bubble System automatically ensures type-safety because a bubble (object) can only be dragged on a slot (parameter) of the correct type. This is because, you can only reference slots which you can see, and you can only see slots which exist. This feature is an advantage over writing text code, which can include hard to detect bugs.

When dragging a bubble, the slots that the bubble is being dragged into (to change the accepting slot’s value) will become larger and highlighted to make it easy for the creator to see which slots will accept the bubble. Only specific slots can receive a bubble. If a bubble is a mismatch for a given slot, this slot will become smaller and move away from the bubble being dragged with an animation looking like a magnet repelling another magnet (because the type of the bubble does not match the type that the slot accepts as value). Showing this visual cue and blocking type mismatch is a feature of visual coding only. It makes it easier to build 3D graphics.

The Bubble System automatically updates the visual change of slot fill, the creator immediately sees the output. This means that dragging a bubble into a slot of the wrong type is caught immediately. Seeing the output immediately after a change is

especially advantageous in comparison to conventional text code, where code is spread across many files, folders and libraries.

6.2.5 Parameter Order Mismatch Solved with 3D interface

In the Bubble System, the user drag and drop to set the value of a slot. The Bubble System 3D interface does not require parameters to be passed in a set order, since parameters are visual. Further, by specifying the name, rather than order, this puts an extra ‘check’ for the programmer, right in front of their face where they are making the change. Additionally, Bubble auto-updates the graphics when slots are filled, so if there is an issue with the code like a number being put into a color slot, the programmer sees it immediately because the visual output is wrong. There is no “blind coding” or uncertainty about the output of the program, because the code is in the same environment as the output of the program.

6.2.6 Misspelling parameters and variables

Sometimes textual code has variables or objects which are named similarly. Visual code helps distinguish these objects, since they are shown using visual representation; also helps show type of the object). When these bugs create Type-correct code, they can be impossible to fix, if it happens in a large system of code, it literally may never be found.

6.2.7 Unfilled parameters solved with 3D interface

If the user forgets to fill a slot (parameter) they know it because it is shown visually that it is unfilled. They see the slot which is unfilled highlighted and empty (unfilled) instead of not knowing what is causing code to not work. Sometimes these bugs will take *hours* to solve in regular text code, since the code might still run, and there might be no errors shown, but there still is a bug.

6.2.8 Visual Syntax

The wave graph structure of the program and its moving parts shown visually gives the programmer a visual representation of the code at all times, for seeing the structure, acting as a constantly updating debugging mechanism, while also acting as the syntax of the code. The Wave Graph is used to visualize logical structure of code, to catch structural bugs preemptively, minimizing the complexity of the debugging process, and allowing for greater expression due to less time spent on debugging, and more energy spent on creating structures of higher complexity than what would be possible with conventional text coding and an ordinary debugger.

7. Building in the Bubble System

7.1 The Bubble System

Bubble is a 3D graphic system software that bridges the gap between visualization and expression, allowing programmers and creators to build three dimensional VR applications. The Bubble System has programming capabilities for building backend structure, using directed graphs to visualize events and code similar to what has been done with CUBE (Najork) and other Visual Programming Systems.

In addition, Bubble can present real time data from different data sources and show their potential interaction. We demonstrate this further below, by showing the association of the time of Elon Musk’s tweets and the value of the cryptocurrency ‘Dogecoin.’

The Bubble System provides pre-built building blocks, called “bubbles” that the user can use to build their own structures. The provided building blocks “bubbles” are built using JavaScript. The system allows creators to build visual and logical structures similar to neurons in the brain and circuits. Programmers simply manipulate the “bubbles” in a 3D environment, instead of writing textual code. Bubble can be used with a VR headset, or simply using a desktop/laptop computer.

The 3D software allows for visualization and zooming to traverse the data because we can use our whole space as the ‘screen’ and never run out of space. Below we describe the process of building using the Bubble System and show the capabilities of the 3D graphics system, the user interface, the possibility of cross-data visualization using real-time data. We also provide an example of a 3D output.

We created a mockup of an app showing data visualization of a cryptocurrency price over time. This is a representation of what a VR app would look like if it was built in the Bubble System, to show how easy it is for the user to build an application using Bubble. We show various features that can be implemented using the system.

7.2 Bubble: Basic Visual Data Structure

7.2.1 Visual Programming

As described previously, visual programming is a ‘divide and conquer’ approach because the creation of the ability to zoom in and out of the code to visualize its

structure and output allows the user to visualize and program on far higher levels of abstraction and generality than using text code. The indirection in visual code allows for visually shown abstractions; in this case instead of having an equation for a sphere, we have its name and physical representation. Text code does not allow for zooming in and out of code and has no visual representation. Also, the Bubble System shows data lazily in space to make it easier to traverse, and not every level of generalization must be shown at once, which makes it easier to build, and also easier to use the software. The visual construction of namespace is important as it allows for granularization of the function : each line of code is instead an object (i.e. bubble/slot/wave) that can be recursively deconstructed.

7.2.2 Visual Data Structures

Visual representations of traditional data structures allow for an avenue for translation from text code to Bubble. The syntax of visual programming could be thought of as these visual data structures. Visual 3D code structure might be thought of as a change in code typography to allow for a different syntax where 3D visual on top of text code for easy manipulation and visualization. Below are some simple diagrams of visual representations of data structures: ones that are analogous to conventional data structures used in text coding currently.

7.2.3 Visual Namespace of Bubble objects

One of the first and most important ideas novice programmers learn is the idea of “namespace” -- that if a variable is defined within a context, such as a method, for example, the variable can only be referenced within that context.

The namespace for variables is commonly represented in conventional text code using brackets or indentations. Brackets are one of the structural components of namespace. Then, we create a visual representation of namespace, to make the idea and essence of namespace both easier to understand and easier to use for the expression and creation of structure. The idea of a bubble is to enclose a namespace visually. A bubble is thus a visual representation (e.g., a circle) of a namespace as a lingual unit which can hold visual representations further within its own structure.

The idea of using a graphical representation to represent code is to replace the brackets in conventional code, since those brackets represent the boundaries of the namespace being referenced. In addition to replacing the brackets with visual spheres, ‘bubbles’ all of the other textual elements also are replaced with a visual syntax.

The Bubble system uses “bubbles” which are spheres representing code objects and visual objects. Figure 2 below shows a bubble with a name (which is also called “Bubble” but could be named anything), showing how names appear for bubbles.

The namespace for variables is enclosed using brackets using conventional text code.

Figure 2: A ‘bubble,’ visual representation of an object



The translation of the Figure 2 into text code would be

```
let Bubble = {};
```

The representation of an object as a sphere introduces the idea of visual namespace and the concept of visual code expression. This is commonly used in many UML diagrams as well, which currently allow programmers to visualize object oriented structure.

7.2.4 Slots: Visual Parameters of Bubble objects

A **bubble** (class/object) is a visual representation of a class/object code, which the user manipulates using the 3D user interface. A bubble is a 3D object that can be dragged to change its position. For example, you can drag bubble 1 into a slot (parameter) of bubble 2 to change the value of that slot to the value of bubble 1 and bubble 2 now has a slot whose value is bubble 1.

A bubble’s underlying structure can be seen by viewing its slots which contain waves (lines of code/information) attached to the slots.

A **slot** is a parameter of a bubble. Slots are functions since when a slot is filled, the waves attached to the slot fire. A slot is a 'method' of the bubble that it is in. Slots fire when they are filled, and waves (similar to lines of code) which are attached to slots (methods and functions of classes). See Figure 3 below for a bubble (named Bubble) with a slot.

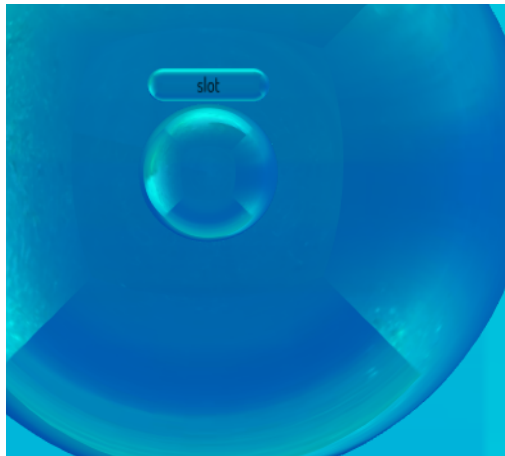
The translation of the Figure 3 into text code to create a new bubble with the name 'Bubble' and a slot (field) set to null. Figure 4 below shows a close up of a slot.

Figure 3: A bubble named “Bubble” with a slot



```
let Bubble = { slot : null };
```

Figure 4: Close Up of a Slot



7.2.5 Changing Slot Value

The Bubble System feature of Dynamic Update, meaning that the output of the program is shown immediately, similar to the Excel feature where all cells update when data is entered. The value of a slot can be changed by filling a value into the slot, or using a visual interface; for example, a number slider. Changing the value using a number slider for the X position slot in the ‘sphere’ bubble changes the value of the sphere’s X position, immediately when the slot’s value is changed.

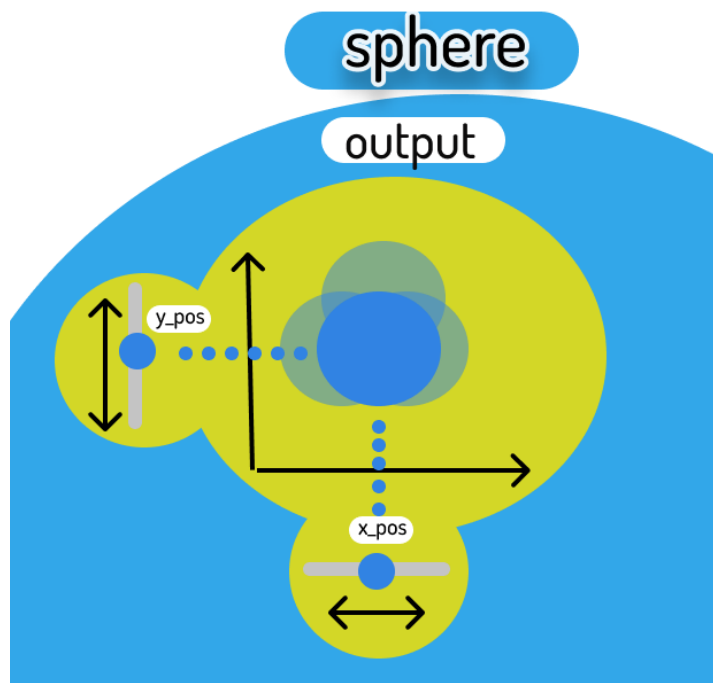
See below the sphere bubble (Figure 5) with two slots:

x_pos, for changing the X position of the sphere;

y_pos, for changing the Y position of the sphere.

The ‘output’ slot shown inside the ‘sphere’ bubble shows the visual output created from changing the value of one of the slots of the ‘sphere’ bubble.

Figure 5: Changing Slot Value



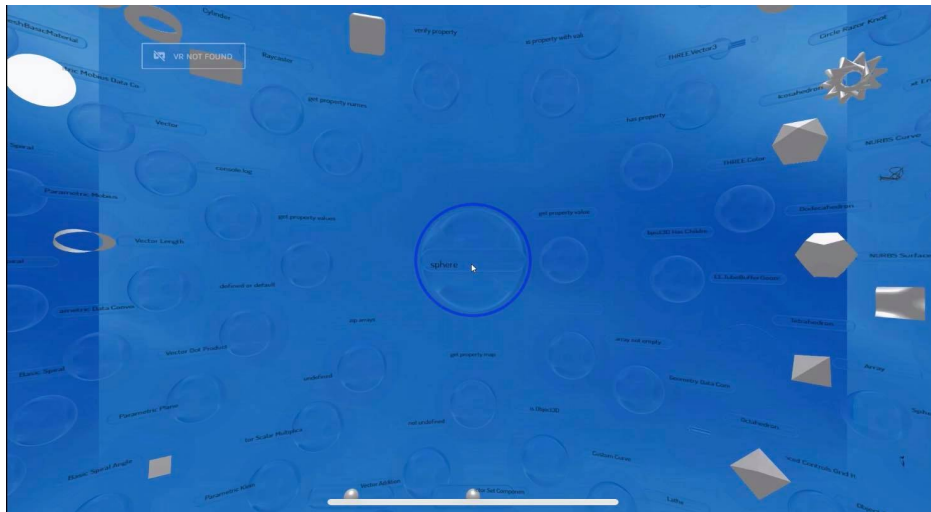
7.3 Biblo Codebase

With the Biblo visual codebase, creators can reuse their structures and visual objects by dragging and dropping bubbles. Creators will never be forced to rewrite and rebuild their structures. They can reuse their creations (‘bubbles,’ their visual

building blocks) by dragging and dropping, making interfacing between different parts of the structure and interfacing between different parts of the system free of bugs, since the code has all been pre-written. Thus, the Bubble system will substantially reduce the occurrence of bugs. It will include a basic codebase for creators to use as their initial building blocks.

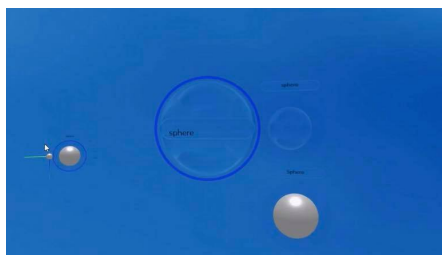
The codebase is a code library of visual code objects, stored and visually represented as bubbles (visual representations of objects.) To search the codebase, the creator clicks on the search bar in the center of the screen and types in the name of the desired bubble (visual/code object) to start a search. If the creator wants to search the codebase, the creator types in the name of the bubble that they are searching for. In Figure 6 below, the creator typed in ‘sphere.’

Figure 6: Codebase Search



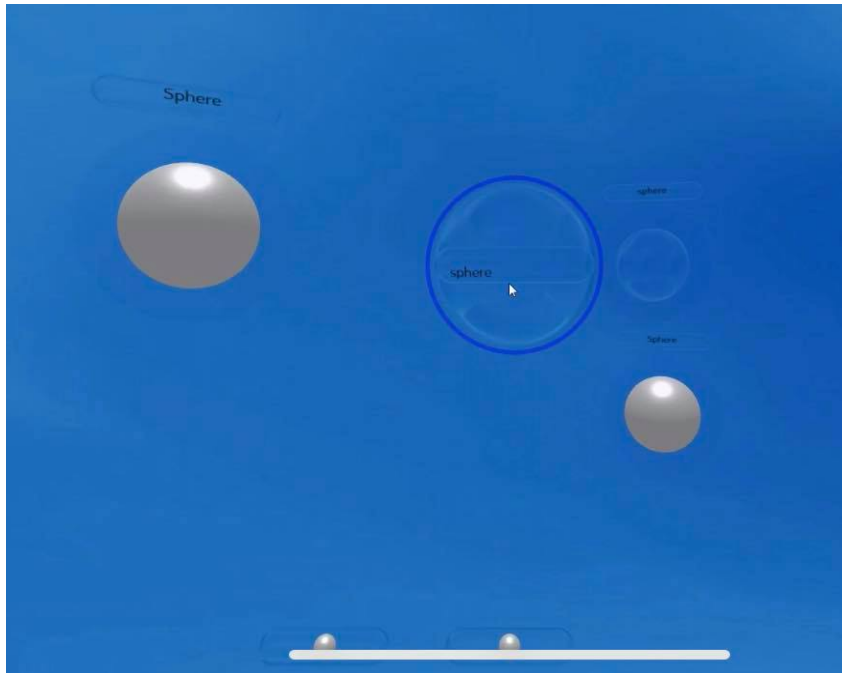
In Figure 7 below, the creator has typed ‘sphere’ in the codebase and the ‘sphere’ bubble has appeared in the lower right corner below the search area.

Figure 7: Codebase Search Result



In Figure 8 below, the creator drags the sphere bubble (visual/code object) into open space (similar to dragging a file onto the desktop) to create a new bubble object instantiation, which can now be used and modified.

Figure 8: Codebase drag search result to ‘Open Space’



If the creator for example wants to change the color of the sphere (changing its value). The creator/programmer:

- Searches the codebase again, this time for ‘black’. One of the results of the search will be the ‘black’ bubble
- Drags that ‘black’ bubble into open space
- Scrolls through the slots of the ‘sphere’ bubble to find the ‘color’ slot of the ‘sphere’ bubble. Note: the ‘color’ slot will only take in a bubble of type ‘color’
- Drag the ‘black’ bubble into the ‘color’ slot of the ‘sphere’ bubble

These actions set the value of the ‘color’ slot of the ‘sphere’ bubble to black; in other words the sphere is now the color ‘black’.

Note that the creator can determine the amount of slots, which they can import into a bubble from the codebase. (There would be a slot called color which is stored in the codebase as a bubble object.)

7.3.1 Bubble Inheritance

In the Bubble System, the creator can search through bubbles in the visual codebase and create a new instantiation of a bubble object by dragging it into open space (like dragging onto a desktop). With this new instantiation, the creator can edit the graphic for that new bubble, adding graphical elements or changing the name of the bubble. Thus, the polymorphism of bubbles is shown visually through changes in the graphic of new instantiated bubbles. Their names can be changed after being instantiated, showing their difference from the bubble that they are inheriting from.

The creator can then change the values of the slots within the instantiated bubble, to further change the properties of the new bubble. Further below we will show an example. of using the codebase to create a new instantiation of a 'sphere' bubble, and change its color slot to 'black' which then changes the 'sphere' bubble instantiation color to black.

7.3.2 Modularity

This system for instantiating and editing the properties of bubbles described above creates a modular system. All of the parts of the system are stored as bubbles which contain slots, and these slots can be updated without writing additional code, which means that there is less overhead created when changes are made. Each change made to the value of a variable or setting parameters can be done with a slot fill of a bubble dragged from the codebase.

7.4 Useful Terms for building more complex structures

Below are some useful terms for building code using bubbles (objects) which contain slots (fields/functions) which have waves attached to them. Each wave is a visual code unit. A wave graph is a directed graph where each wave only fires once the wave before it has fired.

Generalization: Describes the specificity of the data/graphics being shown. Changing generalization level means changing the specificity or level of zoom that the user is observing the data with. A generalized graphical representation allows for viewing data/code/graphics on a higher level of abstraction, by covering certain details with a graphic, allowing the user to view a higher level of abstraction and structure, with less detail. Generalization can be shown in the form of a graphic

representation instead of the fully disambiguated version of a bubble with all of its slots and waves

Zoom in: The user sees a more detail, specific and granular subsection of the dataset. Zooming in is analogous to having binoculars on a dataset. The user gains the ability of viewing the same dataset, but on a more granular level, or lower level of abstraction.

Zoom Out: The user sees a more general subsection of the dataset. This is analogous to zooming out in google maps; makes data more general. The user can see less bubbles but see a larger dataset, zooming out is like being in a satellite where you can see the whole earth from afar, see 'higher level' of abstraction.

Wave: A statement which can be attached to a slot, which then fires when the slot has been filled. A wave graph is a directed graph, and a wave can only fire when the wave which points to it has already fired. When a wave fires, then the waves after that wave (waves that are attached to that wave) go next (See Figure 9 below).

Wave firing: when a wave executes it fires, a wave can only execute when the slot it is attached to is filled/fired.

7.5 Wave Graphs (Directed Graphs) for Visual Code

Waves are used for data passing in the Bubble System. A wave can execute a slot fill, which changes the value of a slot by filling it with a bubble, or changing the value of the slot that the wave is attached to. Waves are shown visually attached to slots, and create a directed graph of chains of waves. Waves execute when the slot is filled that the wave is attached to, similar to a method running when its parameters have been filled.

7.5.1 Wave Graph Example

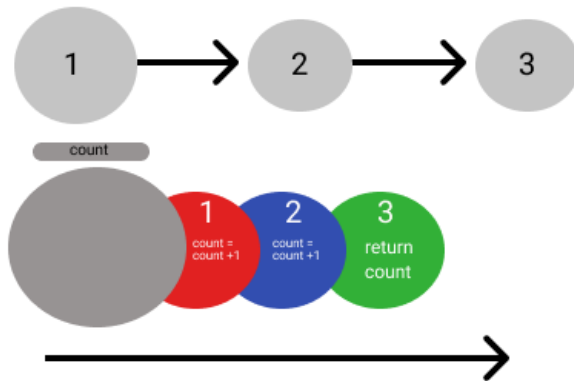
Figure 9 below shows a wave graph as a visual representation of a directed graph. When the slot (parameter) is filled, the red wave fires, since it is the first wave attached to the slot. When the red wave is done executing, the blue wave fires, since it is next in line.

7.5.2 A Directed Graph

In the **directed graph (Figure 9)** shown below, the first row of grey circles represent nodes. The '1' in the grey circle is the first node and it points to '2' which points to '3'. Each numbered grey circle represents a node of a directed graph.

In the second row of Figure 9 below, the colorful circles are waves. The '**count**' circle, in grey, is a slot.

Figure 9: Wave Graph Example



7.5.3 Execution Order

Waves, shown in Figure 9 above, execute in the exact same way:

First the slot (grey) must be filled, then the first wave (red) fires, then the second wave (blue) fires, then the third wave (green) fires.

7.6 Bubble Visual Syntax Rules

For the purposes of the mockups shown below we will show bubbles, slots, and waves in the following colors:

Bubbles: shown in **blue**

Slots: shown in **yellow**

Waves: shown in **red**

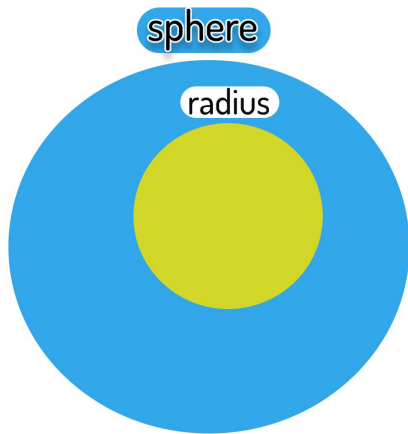
7.6.1 A Bubble with a Slot

Figure 10, below, shows a **bubble**, called '**sphere**.' The '**sphere**' has a **slot**, called '**radius**'

The equivalent syntax in textual code:

sphere.radius

Figure 10: Slot in a Bubble



7.6.2 Filling a Bubble into a Slot

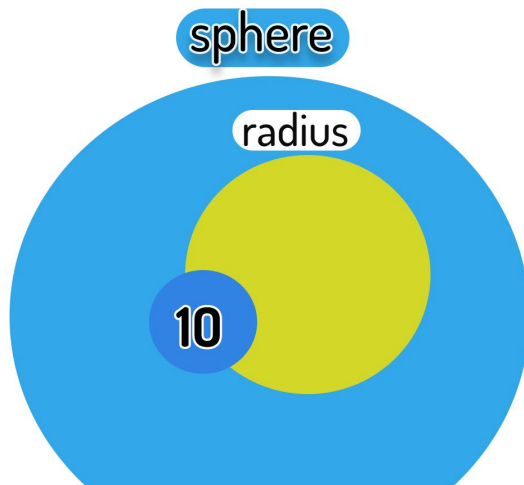
Figure 11 below shows the process of changing the value of a slot, which in this example means changing the value of the radius slot to 10. To do this, the radius slot needs to be filled with a bubble with '10' as its value (numbers are stored as objects.)

A **bubble**, '10' is shown in **blue** on top of the radius slot; the **radius** slot is set to 10.

The equivalent in text code is:

Sphere.radius = 10

Figure 11: Value Setting for Slots



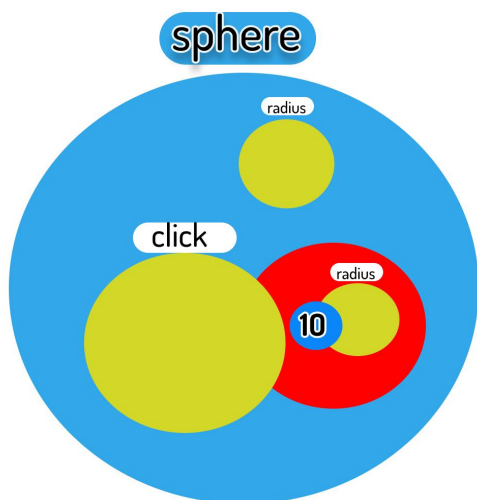
7.6.3 A Slot with a Wave

A wave fires when the slot it is attached to is filled. In this example, the **red wave** is attached to the **yellow slot**; the **red wave fires** when the **yellow slot** is **filled**. In this example the **yellow slot 'radius'** is filled with the '10', a bubble (object) as its **value**--so when the **'click' slot** is filled, the wave will fire, changing the value of the 'radius' slot to '10'.

In Figure 12, below, the red wave attached to the **yellow radius slot** says:

When the yellow **'click' slot** is filled, set **radius** to '10'

Figure 12: Value Setting with a Wave



7.7 Bubble with Slots and Waves

Recall that waves attach to slots, and slots are inside of bubbles as their methods, functions, and fields. In this chapter we show steps on how to use visual code to create a graph of a stock price over time. We illustrate the steps with the figures 13-16.

Figure 13, below, shows as an example a 'sphere' bubble with multiple slots. It is analogous to a class with multiple methods, in conventional coding. Just like a cell, it contains parts which can be broken down individually.

Figure 13 below includes the following new 'sphere' bubbles:

Output: visual output of the bubble, in this case, creating a sphere.

Time: a slot which accepts a value from time data which programmer inputs into time slot

x_pos (X Coordinate): slot in the sphere bubble (object), which changes the x coordinate of the sphere's position. (In this case, changing the positions of many spheres creates a graph, where the x position is set to time data from stock data being passed in)

y_pos : (Y Coordinate) slot in the sphere bubble (object), which changes the y coordinate of the sphere's position. (In this case, changing the positions of many spheres creates a graph, where the x position is set to price data stock data being passed in)

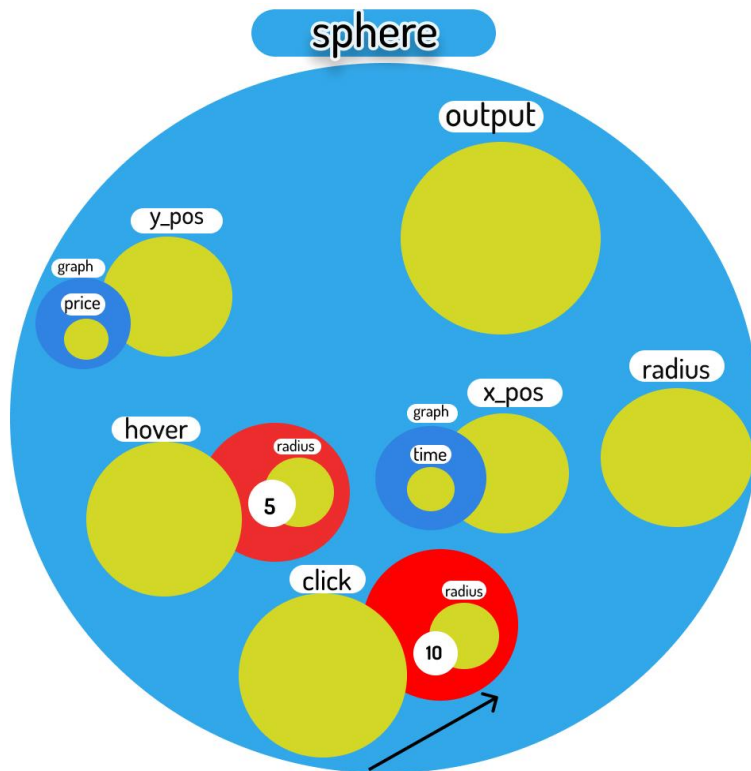
Hover: a slot (parameter), which fires when the user hovers over the sphere; 'hover' is an input handler

Click: a slot (parameter), which fires when the user clicks on the sphere; 'click' an input handler

'5': a number, the value which the radius slot is set to in the red wave attached to the 'click' slot. Numbers are stored as bubble objects.

'Hover' and 'click' are slots which fire when they are set with a value. In this example, the 'hover' or 'click' slot would be filled and fires when the user hovers or clicks on the object. Then its waves would execute after the slot is filled. The 'output' slot is the visual output.

Figure 13: The “Sphere” Bubble with Slots (waves attached to slots)



7.7.1 The 'Radius' Slot

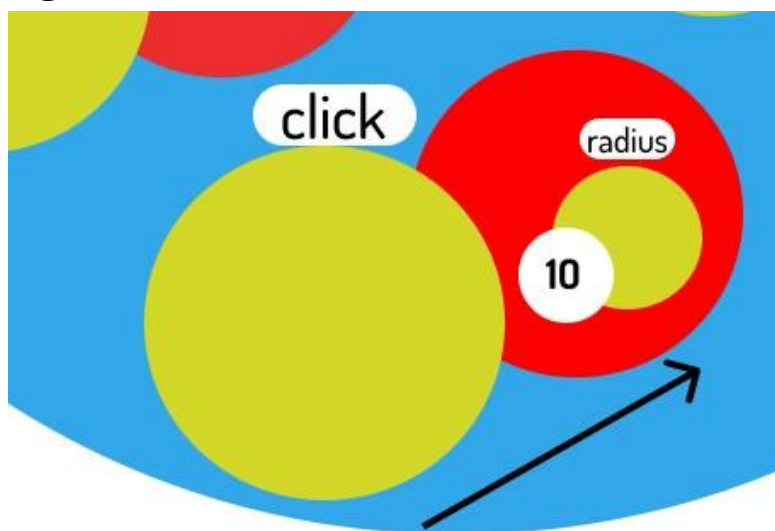
The red circle in Figure 13 above is the 'radius' slot, which is filled when one of the 'hover' or 'click' slots are filled. When the slot is updated with the value '10', the 'output' slot changes, appearing on the 'output' slot. The 'output' slot is the graphical output of the bubble. See Figures 14 and 15 below for zoomed-in details. The 'radius' slot's value will change when it is filled. In this case, its value is '10' because the 'Click' slot has been filled. As shown below in Figures 14 and 15, one of the ways the value can be changed is through a wave attached to a different slot, with the wave containing a slot fill changing the value of the 'radius' slot.

Note that immediately after the value of the 'radius' slot is changed, the output dynamically changes, which in this case is the size of the sphere.

7.7.2 The 'Click' Slot

One of the slots that can change the value of the radius slot is the 'Click' slot. A wave attached to the 'Click' slot might contain a slot fill, which might change the value of a different slot. Figure 14 below shows a zoomed in detailed view of the 'click' slot in yellow from Figure 13 above, with its waves attached to it. The red wave shown below attached to the 'click' slot fires when 'click' is filled with a value; if the user clicks on the sphere, this will update the 'click' slot, and the red wave fires. As seen below, the red wave fills the 'radius' slot, and changes the value of the sphere's radius to '10' since the '10' is a bubble object, shown with 'radius' attached to it, filling '10' as the value of the 'radius' slot.

Figure 14: The 'Click' Slot



The translation of Figure 14, the ‘Click’ slot into **text code**:

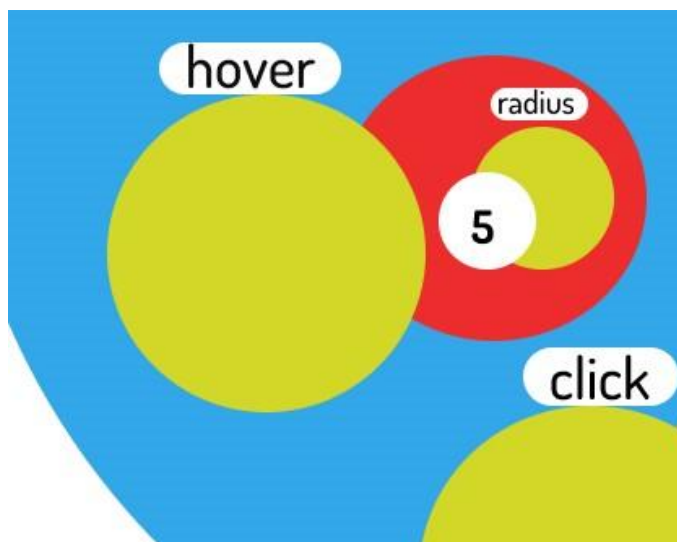
```
click () {  
  
  this.radius = 10;  
  
}
```

7.7.3 The ‘Hover’ Slot

The ‘hover’ slot can change the value of the radius slot. A wave attached to the ‘hover’ slot might contain a slot fill, changing the value to ‘10’ or ‘5’ by filling a slot, as mentioned above and demonstrated below. The ‘hover’ slot is only filled when the user hovers over the sphere. Hover is an input on the sphere object, meaning it senses when the user is hovering.

Figure 15 is a more detailed view of the ‘hover’ slot from the ‘sphere’ bubble in Figure 11 above. In Figure 15, the red wave fires when ‘hover’ is filled. If the user hovers over the sphere, the ‘hover’ slot will fire and the red wave will fire, since it is attached to the ‘hover’ slot, and the radius of the sphere changes to ‘5’, since the red wave changes the value of the ‘radius’ slot to ‘5’ as the new value being filled into the slot. See Figure 15 for a closer view of the red wave and this slot fill. One way to think about waves executing slot fills is their utility in being an interface for the equivalent of the dot notation used in text code.

Figure 15: ‘Hover’ Slot:



The translation of Figure 15, the ‘Hover’ slot, into text code:

```

hover ( ) {

  this.radius = 5;

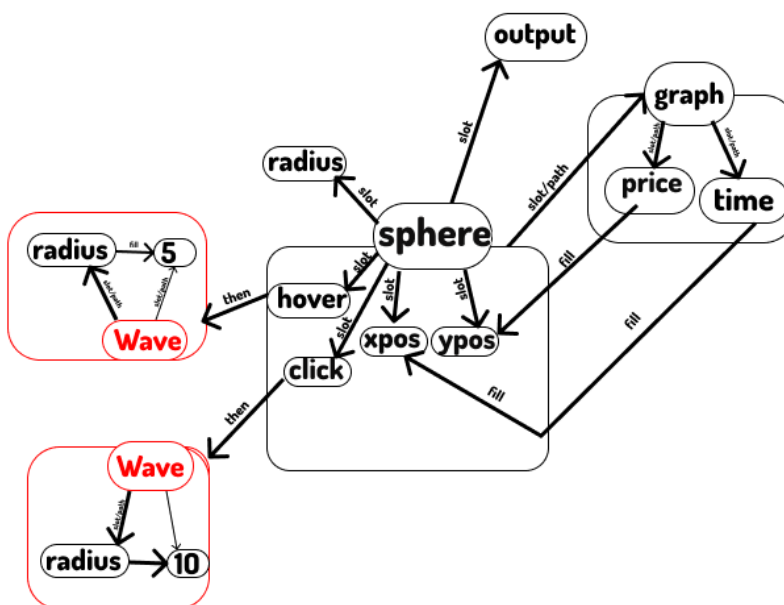
}

```

7.7.4 Directed Graph Form of the ‘Sphere’ Bubble

Figure 16 shows the dataflow of the ‘sphere’ bubble with slots : ‘xpos’, ‘ypos’, ‘click’ ; and ‘hover.’

Figure 16: Directed Graph Form of the ‘Sphere’ Bubble



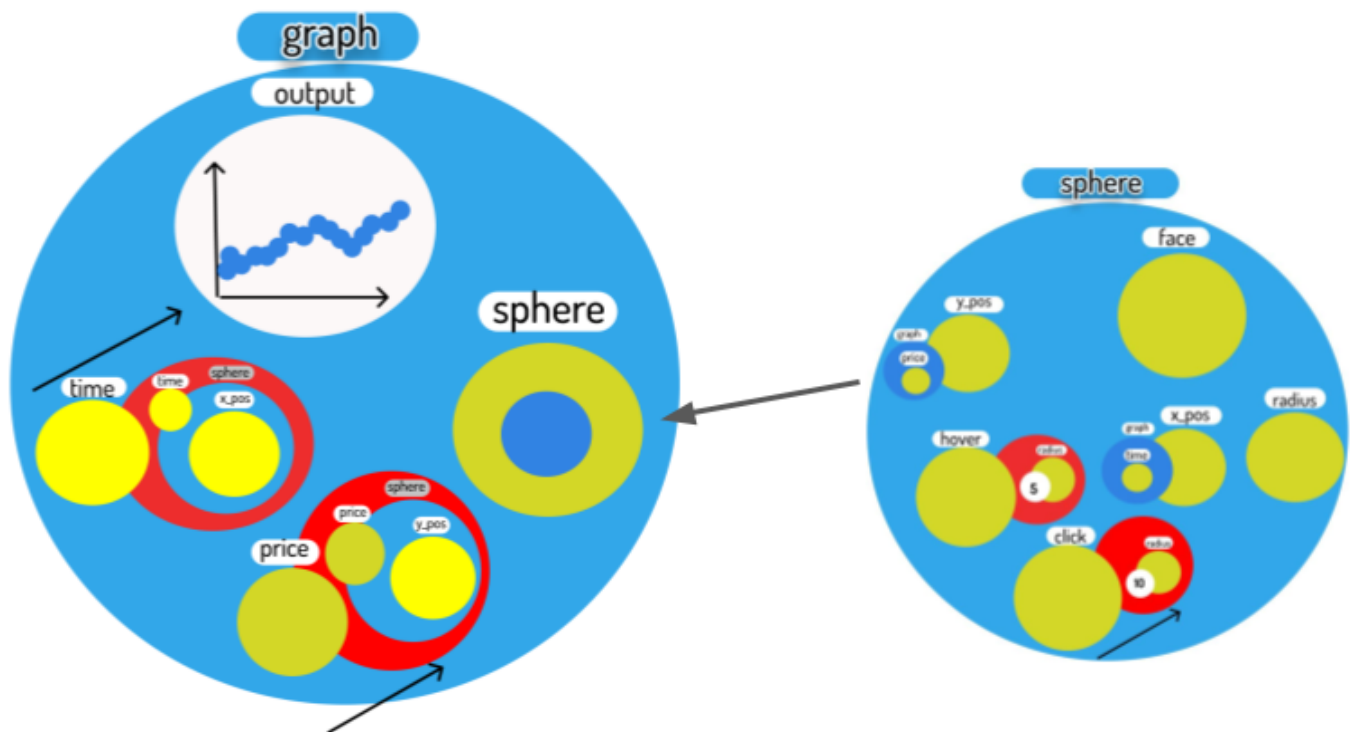
7.8 A More Complex Example using Slots and Waves, ‘Graph’ Bubble

In Figure 17, below, we introduce a small example of visual code to create a graph of a stock price over time. The goal is to show a longitudinal graph of a stock price using a bubble with slots, which then have waves attached as visual coding blocks.

We have an X axis (x_pos) representing time, and a Y axis (y_pos) representing the price of a stock. We show an x position slot, “x_pos” and a y position slot, “y_pos.” Each time point corresponds to a sphere on the graph, and all of the spheres comprise the graph shown in the ‘output’ slot of the graph. The blue dots represent time and price.

The diagram is setting the ‘x_pos’ slot of the ‘sphere’ bubble from above to the value of the ‘time’ slot, and sets the ‘y_pos’ slot of the ‘sphere’ bubble to the value of the ‘price’ slot. The graph bubble contains the sphere bubble, shown generalized with a graphic, instead of the slots and waves comprising its structure shown in the sphere bubble below. See below for the ‘sphere’ bubble and ‘graph’ bubble. These ‘price’ and ‘time’ slots will take in the data from a stock. ***Note that the sphere slot in the ‘graph’ bubble is shown with a graphic, instead of showing all of its slots and waves: see Ch 7.7*

Figure 17: More Complex Example using Slots and Waves Syntax



Equivalent Code

```
Graph {
    double price;
    double time;
    setPrice(double price ){
```

```

        this.price = price;
        this.sphere.y_pos = price
    }

    setTime(double time ){
        this.time = time;
        this.sphere.x_pos = time;
    }

    getPrice(){
        return this.price;
    }

    getTime(){
        return this.time;
    }

    Sphere sphere = new Sphere();
}
Sphere {
    double x_pos;
    double y_pos;
}

```

As shown above, wave graphs provide a simple way to build applications. The above bubble with slots, and waves attached to each slot demonstrates how wave graphs can be used to build applications. A 'price' slot takes in price data, which updates along with a time slot, a parameter which accepts time data. When the price slot is filled, the 'sphere' bubble's 'y_pos' (y coordinate for the position of the sphere) slot changes its value to the value of the price slot. This means that as the price changes, the vertical position of the spheres changes. When the 'time' slot is filled, the 'sphere' bubble's 'x_pos' slot has its value set to the time value from the time slot. This means that as the time changes, the x position of the spheres is updated with the time value, since the x position parameter is set to the time.

The creator could decide to include live updates of the data, for example using stock price data in real time. These updates happen live as the price slot and time slot (parameters) are filled with data. As you will see, 'sphere' is a slot within the 'graph' bubble, and the 'sphere' bubble also shows the variable setting for 'x_pos' (x coordinate for the position of the sphere). We showed the 'sphere' bubble earlier as well, and have provided it above for reference. The example VR application we created using a mockup in the sections 'User Interface for Building 3D Graphics' and 'VR Application' will show a small example of an application created using the visual code from the above example.

Figure 18 shows the 'price' slot, which sets the 'sphere' bubble's "y_pos" (Y position) slot to the price value passed in by the programmer.

Figure 18: Price Slot

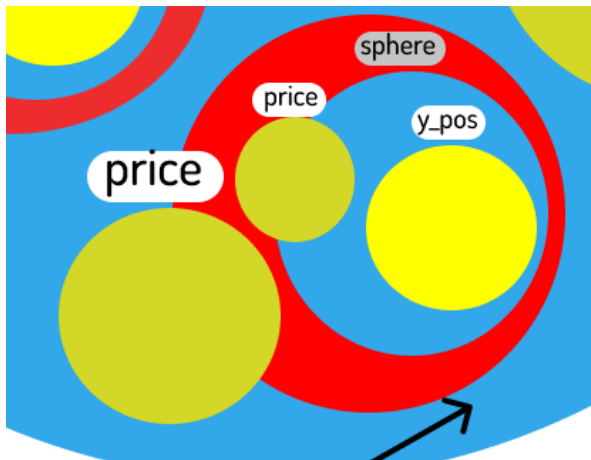
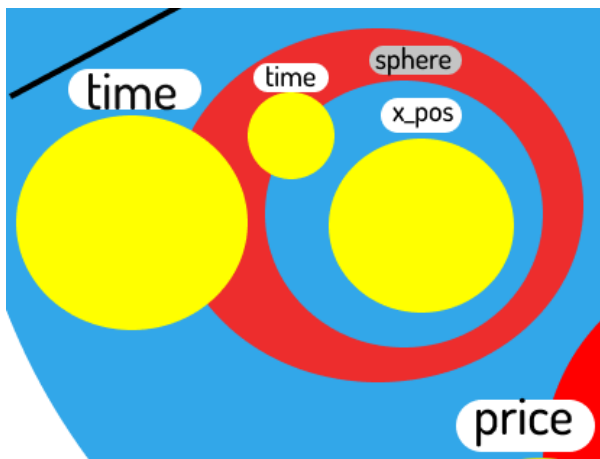


Figure 19 shows the 'time' slot, which sets the 'sphere' bubble's "x_pos" (X position) slot to the time value passed in by the programmer.

Figure 19 - Time Slot



7.9 Advantages and uses of Wave Graphs

One of the uses of Wave Graphs for visualization is showing the paths of different interactions, similar to what Kleyn [9] and Rumbaugh [10] describe (Chapter 3). These graphs show branches of different interactions, and shows the visual changes in each visual representation shown in the directed graph. Using a Wave Graph for interaction paths is similar to what we describe in Chapter 7.4.2, about Type Graphs. Both are different Wave Graphs which can be used for visualization purposes to view changes, both visual and non-visual changes (structural changes can be shown visually by simply showing which slots have been filled, and with the values filled in those slots). For example, if a sphere is added to a scene, the new wave visual representation on the interaction graph would contain the added graphical elements. In the above ‘hover’ and ‘click’ example, the wave would visually show the changes to the visual created by the interaction event ‘hover.’ ‘Hover’ is a slot, and when it is filled, it changes the size of the sphere. Meanwhile, a visual wave attached to the ‘click’ event could show the size change of the radius slot to ‘10,’ while the ‘hover’ slot shows the effect of filling the ‘hover’ slot, which changes the radius to ‘5.’

8. User Interface for Building 3D Graphics

8.1 Example: Data Visualization and Expression in Bubble

The Bubble System is able to show data in a dynamic way and in real time, since the graphical output updates automatically when slots are filled. The Bubble System is able to support the visualization of data from different datasets, which can be imported from the codebase, and dragged into a slot to update the output, similar to the Excel feature where the data shown automatically updates when a variable is changed. However, Bubble is different in that it also allows for importing data in a 3D interface, where the codebase and the coding environment is all in one 3D environment, and there are no separate windows and interfaces. Our example in this Chapter shows the potential effect of Musk’s tweets on the price of Dogecoin, a cryptocurrency. We use the price data of Dogecoin to create a graph of its price over time. This is the dataset we are using for data visualization for the example below.

We show the potential correlation of Elon Musk’s tweets and the price of Dogecoin all in one graph application, illustrating the potential interaction of the data. We choose Elon Musk’s tweets because his tweets on Dogecoin have been known to

precede large increases in the price of the currency Dogecoin during late 2020 and early 2021, as we show in this visualization.

Tweets and other data can be viewed along with the stock price; combining these points is our cross-data visualization application concept. These connections are hard to visualize in other applications, because the user must manually write code to view different datasets separately, sometimes in separate applications altogether. As far as we know, this cross data-visualization feature is exclusive to the Bubble 3D Graphics System.

8.2 Mockup of VR Application

The mockup shown in Figure 20 is a 2D representation of what building 3 D graphics would look like in 3D. In 3D instead of circles, spheres are shown.

The bubble shown in this example of building 3D graphics is a template being used to create 3D graphics. The creator will fill the slots in the bubble to customize the output that is created by the system.

***Note that the ‘graph’ slot in the ‘stock graph’ bubble is mentioned in the previous section, Chapter 7, and it is shown here below with a generalized graphic. See Ch. 7 for a more detailed version of its slots and waves, the visual code behind the ‘graph’ slot.*

8.3 Stock Graph Bubble

Figure 20 below shows an example of a ‘stock visualization’ bubble with different slots, which are its functions and fields, as we mentioned in section 7.6. It has ‘start’ and ‘end’ slots and slots for ‘stock’ and ‘source’ which the user can fill to determine the data shown.

Figure 20 - ‘Stock Graph’ bubble with its slots:

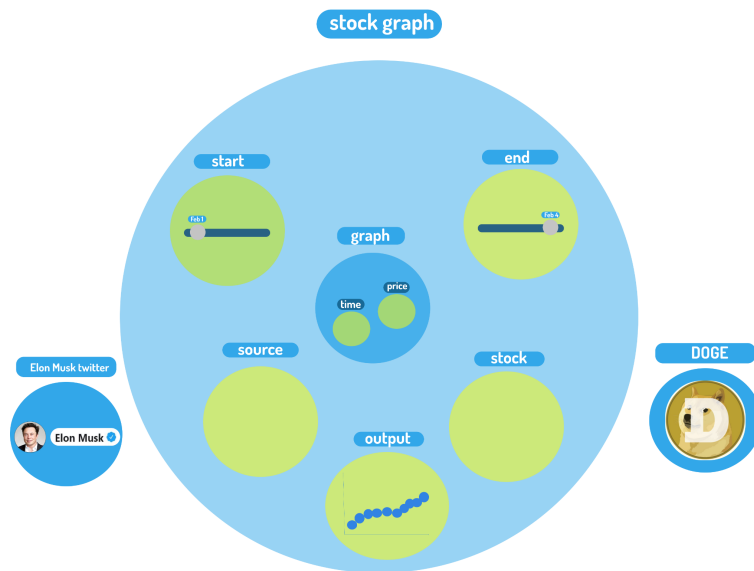
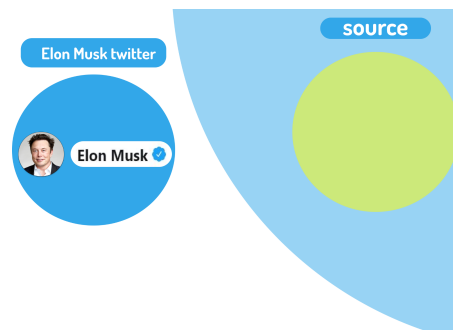


Figure 20, above, the **Stock Graph bubble**, has the following **slots** (parameters):

‘source’: a slot (parameter) which takes in a news source as its value.

In this case, we show the creator dragging in the bubble (object) Elon Musk tweets into the source slot (parameter) of the bubble "stock visualization." This is part of our example dataset for the application, which in this case is Elon Musk’s tweets. The creator can determine the source by filling the ‘source’ slot with a bubble, (as long as the bubble is of the correct type), which they will drag into the slot from the codebase. For this example, the bubble entered into the ‘source’ slot is Elon Musk’s twitter (See Figure 21 below.)

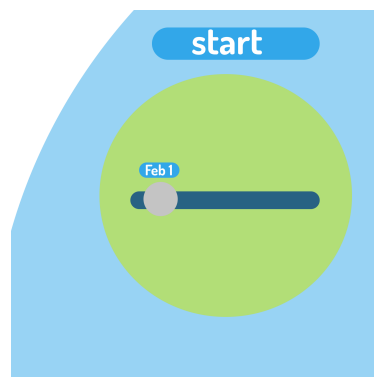
Figure 21 : Source Slot



‘start’ : a start time for the dataset of stock price data over time.

In this case, we use a **number slider** to change the slot’s value. A number slider is an example of a slot type of interface where the user can change the value of the slot by sliding a knob to change the value of a slot, just like sliding a knob to change a volume level in a radio. A number slider can be used in cases where there is a numerical value for the slot. Because we entered the start time ‘Feb 1’ in the ‘start’ slot, we are viewing the stock data starting at ‘Feb 1’ as its value (See Figure 22 below.)

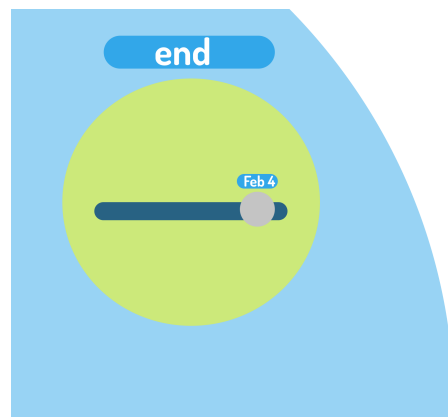
Figure 22: Start Slot



‘end’ : an end time for the dataset of stock price data over time.

Because we entered the end time ‘Feb 4’ in the ‘end’ slot, we are viewing the stock data ending on ‘Feb 4’

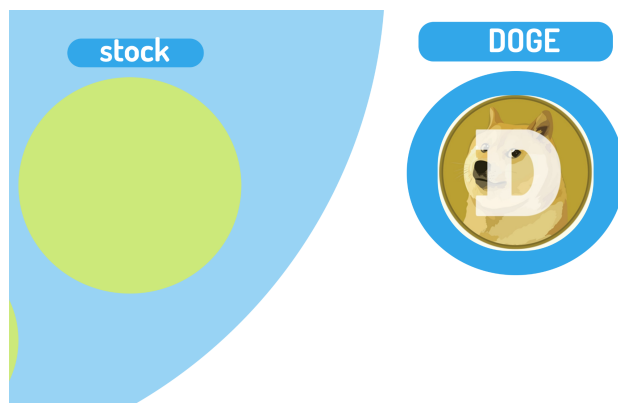
Figure 23: End Slot



‘stock’ : the stock which is being viewed in the application.

The creator can determine this by filling the ‘stock’ slot with a bubble (as long as the bubble is of the correct type) which they will drag into the slot from the codebase. For this example, because we have filled the slot ‘stock’ with the bubble ‘DOGE,’ we are showing Dogecoin’s price data (See Figure 24 below)

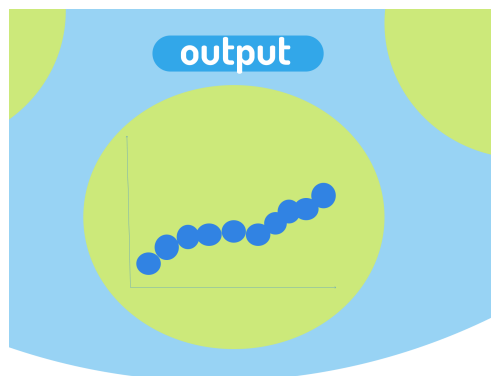
Figure 24: Stock Slot



‘output’ : the visual output of the ‘stock graph’ bubble.

The output slot is the visual output of the ‘stock graph’ bubble and is shown in the same space as the ‘stock graph’ bubble and its slots and waves (see Figure 25.) The feature of showing the visual output in the same space as the code is used with Dynamic Update feature described in Ch 7.8.2, meaning that when a slot is filled, the visual output is shown immediately.

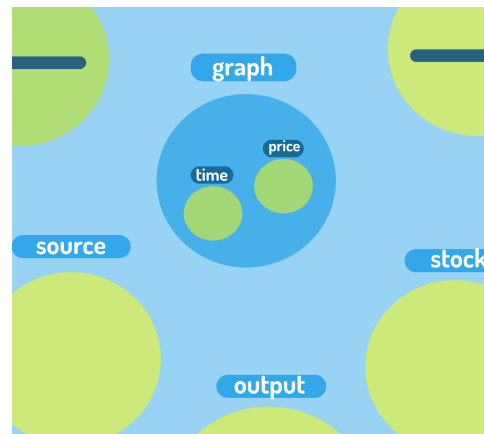
Figure 25: Output Slot



‘graph’ : a slot in the stock graph bubble. Figure 26 is a visual representation of the ‘graph’ bubble visual code (Ch 7.6.8)

This is a visual representation and generalization of the graph bubble with its slots and waves which create the full graph and set the 'price' slot to the 'time' slot, and setting the position of all of the spheres on the graph according to the data entered by the programmer. The graph slot shown here is a visual representation of the fully disambiguated 'graph' with all of its slots and waves.

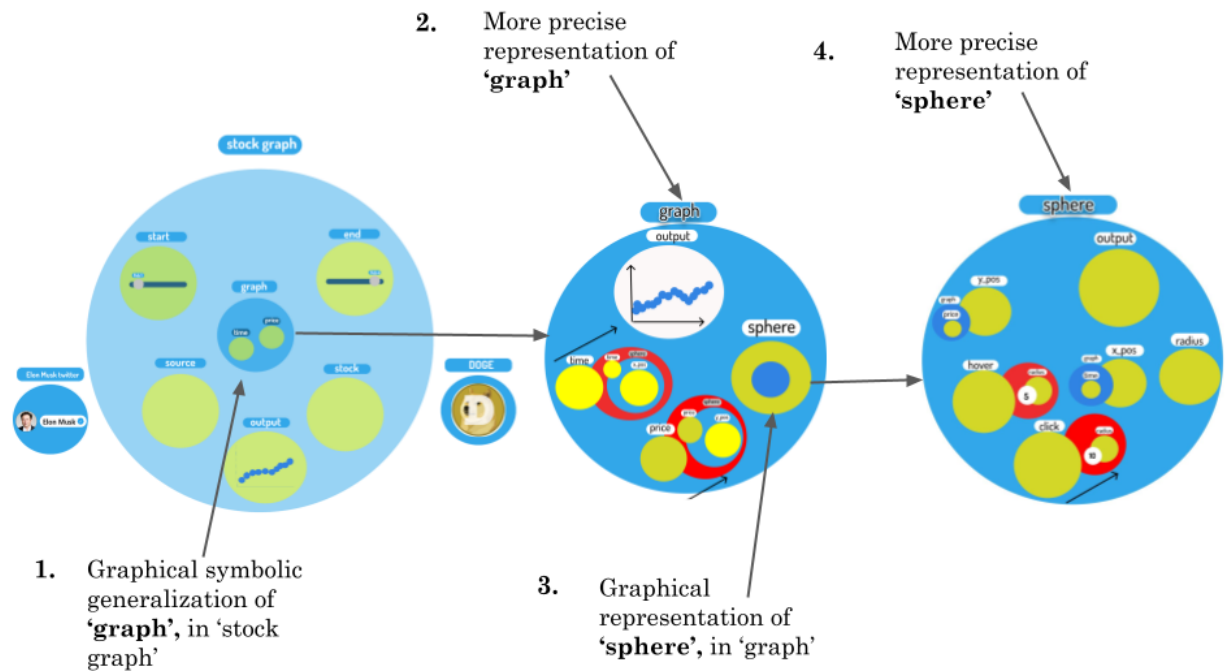
Figure 26: 'Graph' Slot



8.4 Visualizing Abstractions with Visual Code

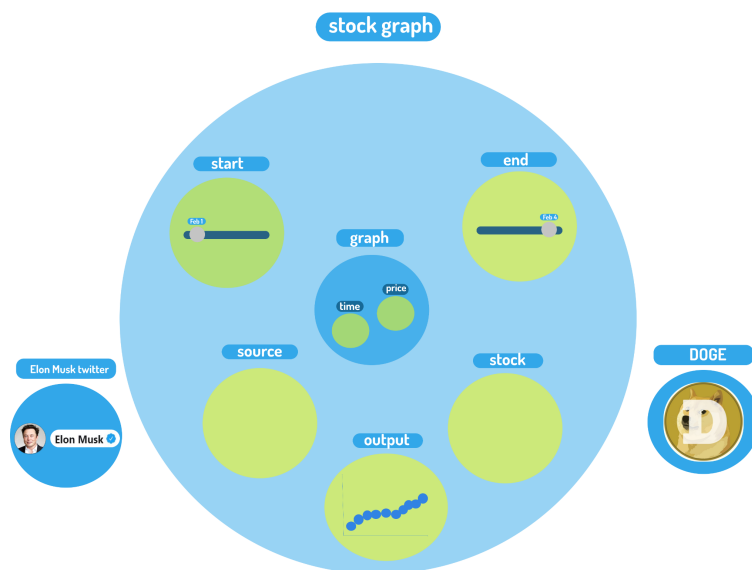
Visual code enables the visualization of abstractions. To review, we created a stock graph application (Chapter 8.2-8.3) using a 'stock graph' bubble.

Figure 27: Abstraction Levels



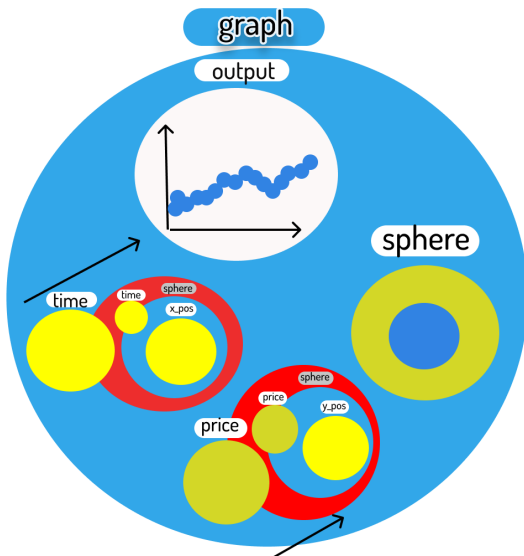
This bubble below contains a 'graph' slot shown through a generalized graphical representation. See below, Figure 28, the highest level of abstraction of visual code:

Figure 28: Stock Graph, Highest Level of Abstraction Visual Code



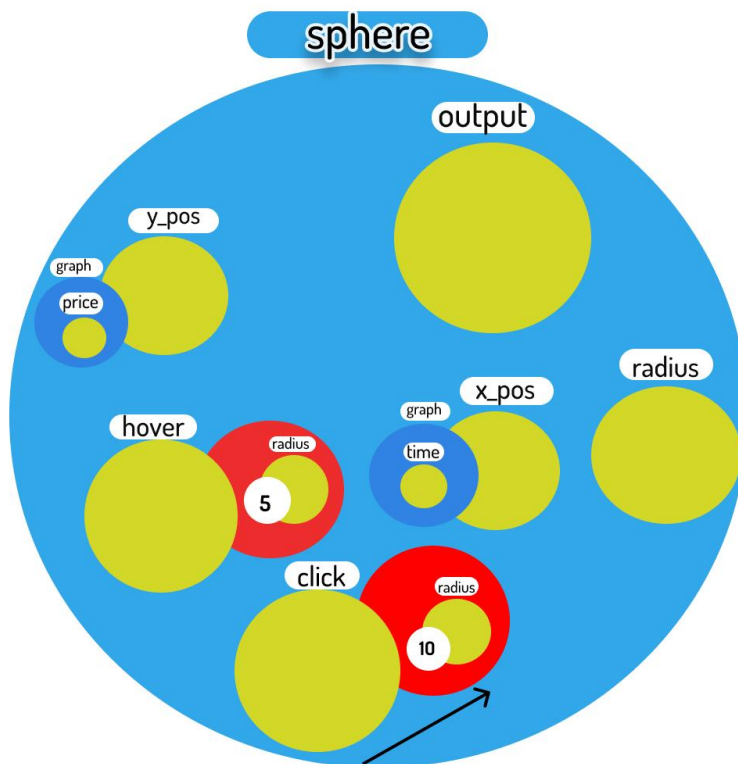
The fully granular representation using slots and waves of the 'graph' slot, Figure 29, a more detailed view of a lower level of abstraction of visual code:

Figure 29: Graph Bubble, Lower Level of Abstraction Visual Code



Finally, we go further to view the 'sphere' bubble inside of the 'graph' bubble, the lowest level of abstraction:

Figure 30: Sphere Bubble, Lowest Level of Abstraction Visual Code



8.4.1 More on Visualizing Abstractions

The ‘sphere’ bubble (Figure 30 above) shows how the Bubble System can be used for showing abstractions visually. Nesting graphics inside of other graphics is the way that the system shows levels of abstraction. Generalized graphics enable the creator to traverse different levels of abstraction using visual cues and gain a deeper knowledge of the structure. In the Stock Graph Bubble, (Figure 28) the ‘graph’ slot contains a generalized graphic and does not show all of the slots and waves shown in the ‘graph’ bubble (Figure 29). The ‘graph’ bubble (Figure 29) contains the ‘sphere’ slot, as a generalized graphic (instead of all of the slots and waves in its structure.) The ‘sphere’ bubble (Figure 30) is shown in more detail with all of the slots and waves. These generalized graphics are indirected using visual representations to reduce the amount of information shown, which then allows for visualization capabilities.

8.5 Technical Details Underlying Visual Code Example Shown

Figures 31-34 below show the underlying structure of the example shown in chapters 7.7 - 8.4 above. In the preceding chapters the diagrams are generalizations of code, using graphical representations as symbols for indirection from the underlying structure and its granular details.

Figure 31 below does not use graphical representations for indirection, it rather is completely precise, and uses the visual syntax strictly.

In Figure 31, the ‘stock graph’ slot contains the slots ‘graph’ and ‘stock’ from the preceding example in Ch 7.7-8.4. However, in Figure 31, the slots ‘stock’ and ‘graph’ both show additional underlying structure using the Bubble System Visual Syntax. The slots ‘time’ and ‘price’ are still within the ‘graph’ bubble but with both ‘plot’ and ‘in’ as slots of ‘graph.’ This is showing that the syntax can be used for more computationally intense examples, and is a strict syntax for expression. The underlying structure written in precise syntax was not shown in the previous examples because it is not required for visual coding, which occurs on a higher level of abstraction, using generalized graphics which abstract from the implementation details of code. Using generalized graphics allows programmers to build structures using symbolic visual representations rather than being required to be responsible for all of the details for all levels of abstraction.

The below Figures 31-34 illustrate how the data will be passed into the system, showing the structure which the system uses for taking in data and structuring it using a visual syntax.

8.5.1 ‘Stock Graph’ Underlying Structure using Exact Visual Notation

Figure 31 below shows the ‘stock graph’ bubble with all of its slots. Note that it still has the same slots, but all of the implementation details including mapping from keys to values for passing in and processing the data have been completed and shown in the most granular format.

Figure 31: ‘Stock Graph’ Underlying Structure

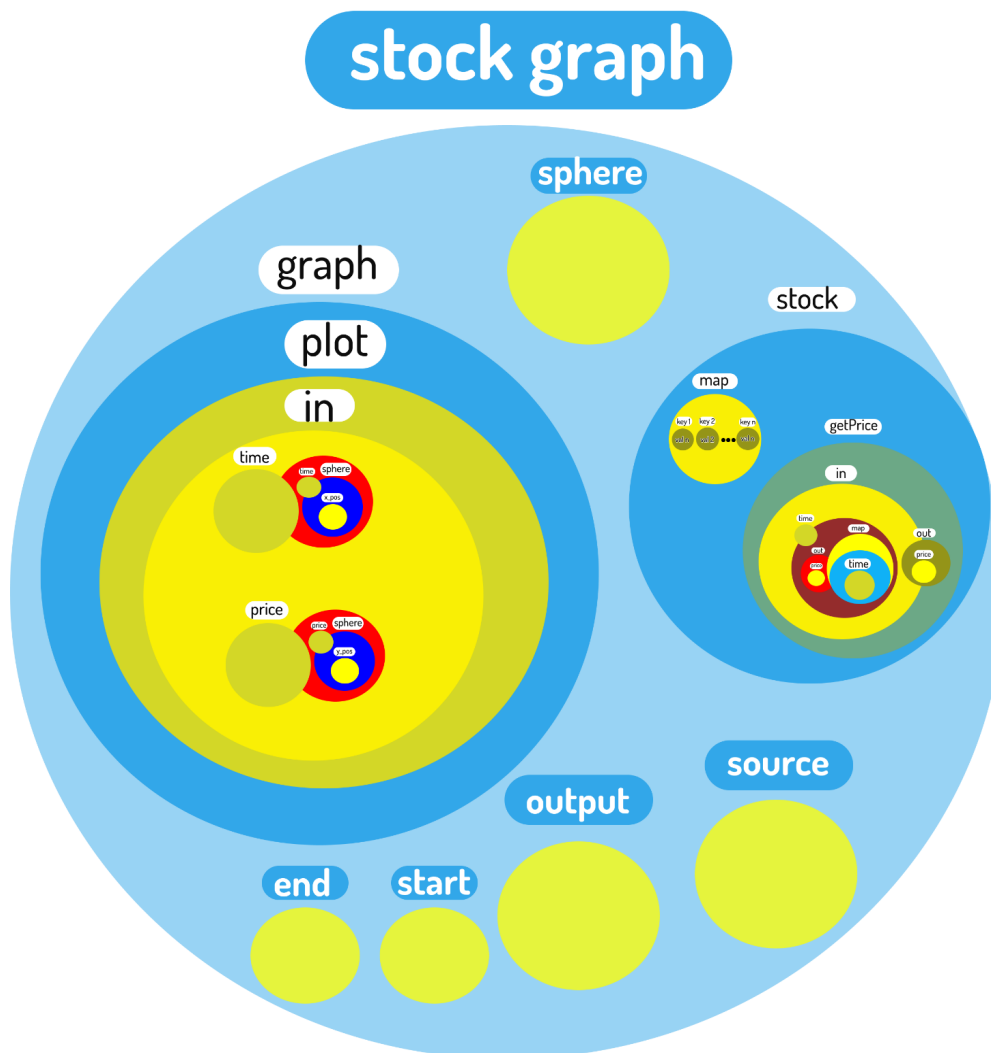
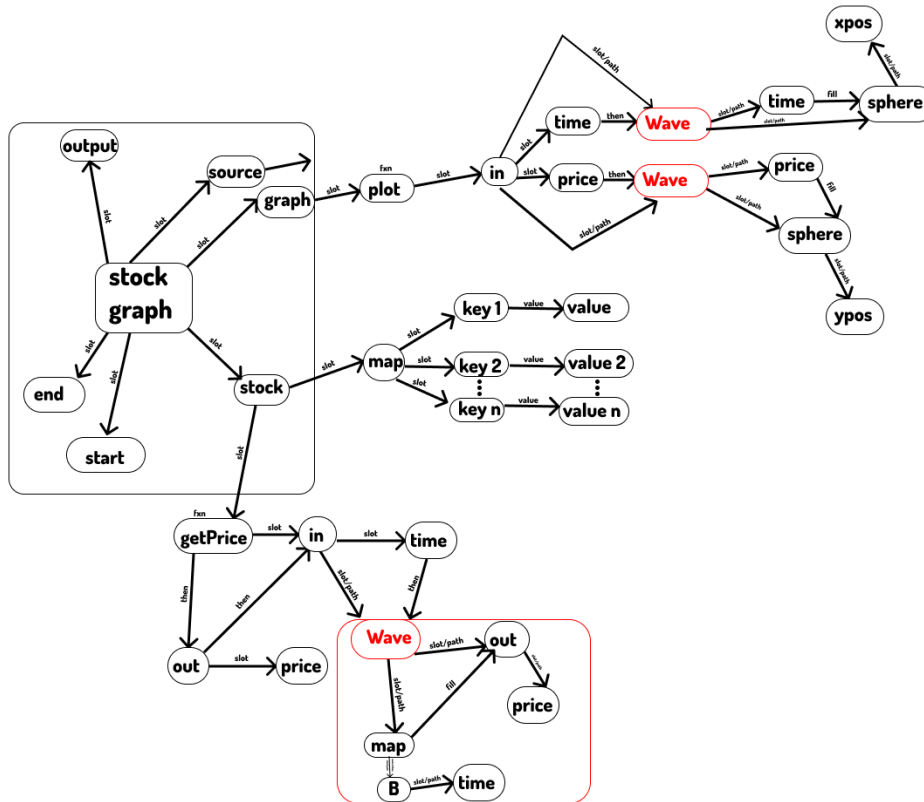


Figure 32 below shows the dataflow of the ‘stock graph’ bubble with slots : ‘getPrice’, ‘stock’; and ‘graph.’

Figure 32: 'Stock Graph' Directed Graph



This is equivalent to:

```

Class StockGraph{
    Private Graph graph = new Graph;
    Private Stock stock = new stock;
}

Class Stock {
    Private Map map = new HashMap(double Date)
    Private Double getPrice(double Date, double time){
        return this.map.get(time);
    }
}

class Graph {

    Private void plot(time : double Date, price : Double){
        This.sphere.xpos = time;
        This.sphere.ypos = price;
    }

    private Sphere sphere = new Sphere( );

```

```

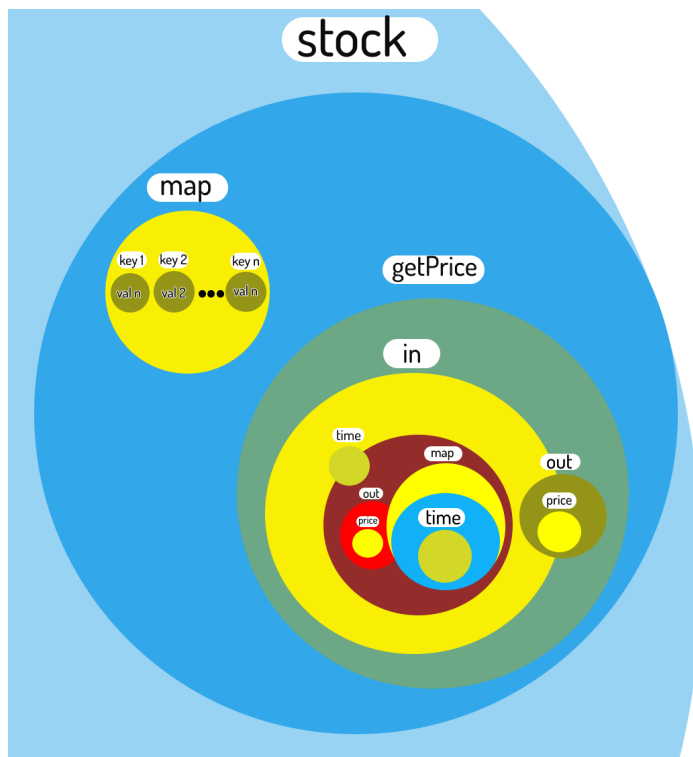
Class Sphere {
    double xpos;
    double ypos;
}

```

8.5.2 'Stock' bubble Underlying Structure using precise Visual Syntax

Figure 33 below is a closer view of the 'stock' bubble inside of the 'stock graph' bubble. It contains the 'map' slot, which filters the data, and maps from keys 'times' to values 'prices.' 'in' takes in 'time' and 'out' returns the corresponding 'price' as output. The 'stock' bubble is inside of the 'stock graph' bubble and contains a 'map' slot, which takes the data passed in and maps from times to prices (keys to values.) The 'stock' bubble contains a 'map' slot, which takes the data passed in and maps from times to prices (mapping keys to values.) The 'getTime' slot takes the data and forwards from time input to price output, using 'map' as intermediary filter

Figure 33: 'Stock' Underlying Representation



This is equivalent to:

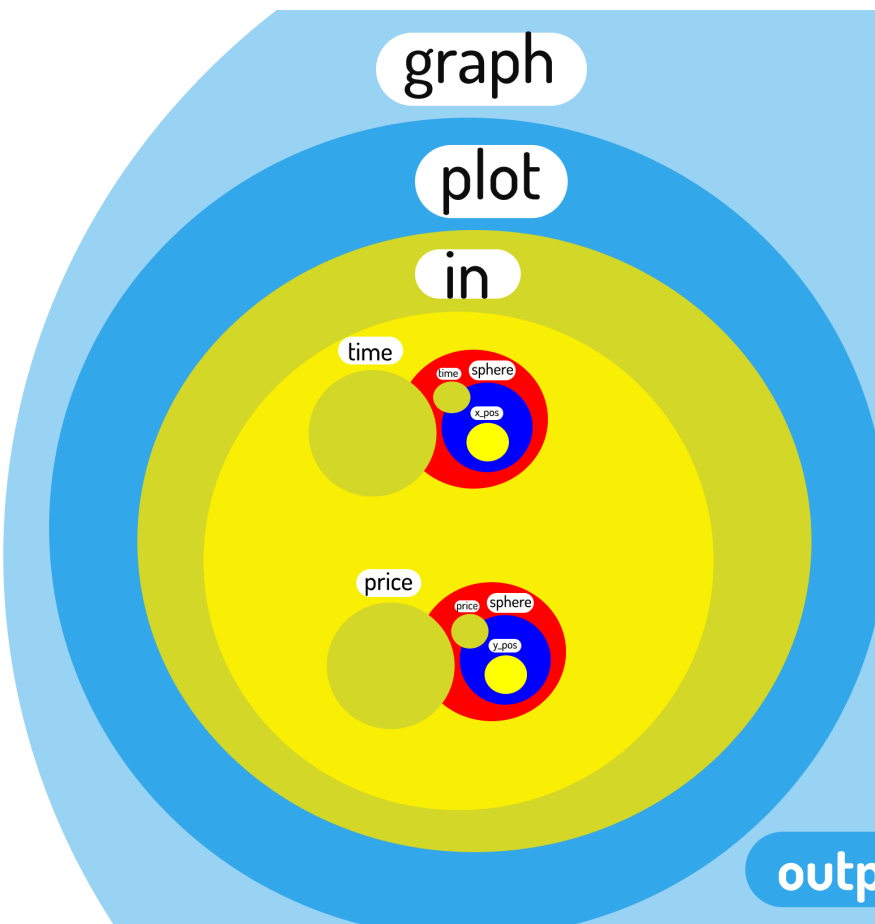
```

Class Stock {
    Private Map map = new HashMap(double Date)
    Private Double getPrice(double Date, double time){
        return this.map.get(time);
    }
}

```

See below the ‘Graph’ bubble which is inside of ‘stock graph.’ The ‘plot’ slot of the ‘graph’ bubble represents a function. The ‘in’ slot of ‘plot’, represents the parameters to the plot function. The ‘plot’ slot has two of these input parameters, ‘time’ and ‘price’. When one ‘calls’ the plot function or ‘fills’ its input parameters, the red waves set the x and y coordinates for the sphere in the ‘sphere’ slot, which in turn updates the position of the sphere in the graph.

Figure 34: ‘Graph’ Underlying Representation



This is equivalent to:

```
class Graph {
    Sphere sphere = new Sphere();
    void plot(double time, double price){
        this.sphere.x_pos = time;
        this.sphere.y_pos = price;
    }
}
```


8.5.3 Bubble Syntax 'For Loop'

Figure 35 below is a disambiguation showing the underlying structure of the symbolic graphic for the 'for loop' in the Bubble System. The 'for loop' used here is technically correct, while the generalized 'for loop' graphic used in Figure 38 further below is the 'for loop' using is a generalized graphic using symbolic visual representation, implementing indirection to hide granular details, like incrementing the variable 'i' and the precise flow of the structure. Slots 'i' and 'n' are slots which are the parameters for the loop.

Once the body of 'for loop' has finished executing, shown in red: the 'for loop' increments: 'i' is a slot attached to 'plus', and 'i' is attached to plus which changes the value of 'i', filling it, and repeating the loop until the 'i' slot is equal to 'n' slot. Note that both yellow slots 'i' and 'n' are slots, which both have the grey oval wave attached, showing that 'i' and 'n' are filled, and then the loop executes.

Figure 35: For Loop Syntax

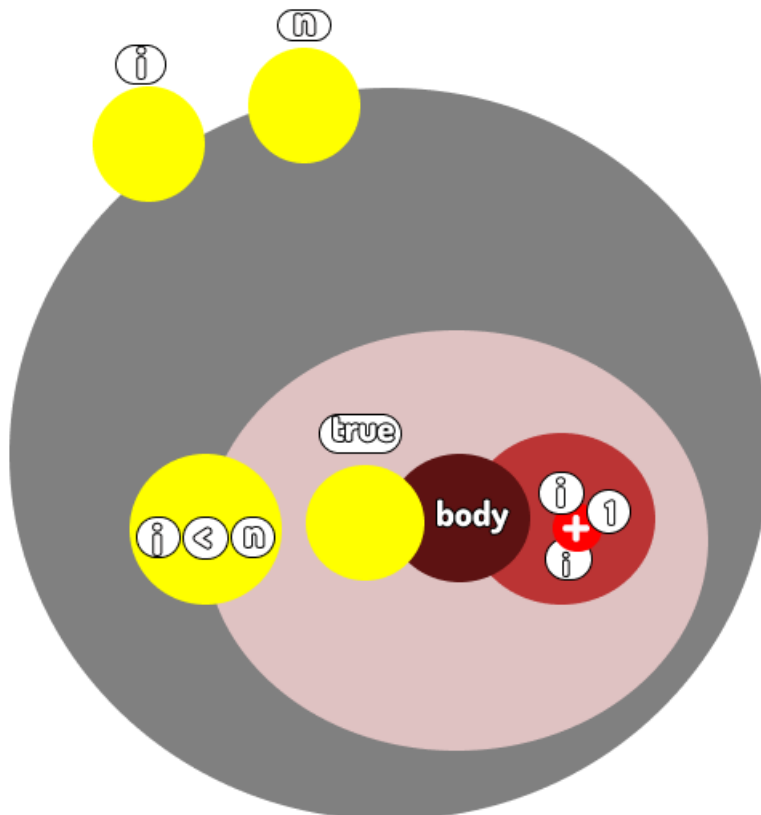
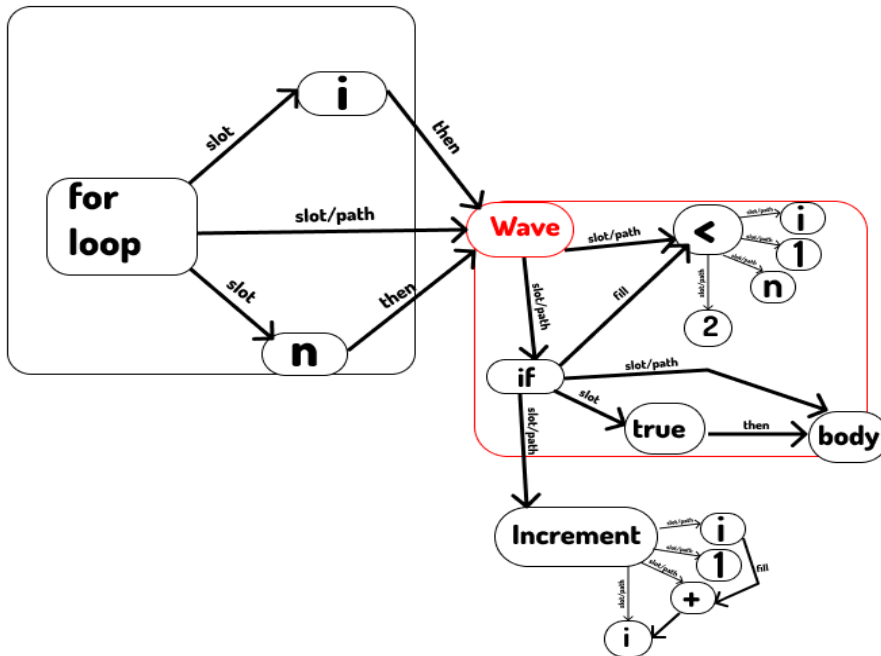


Figure 36: For Loop Directed Graph



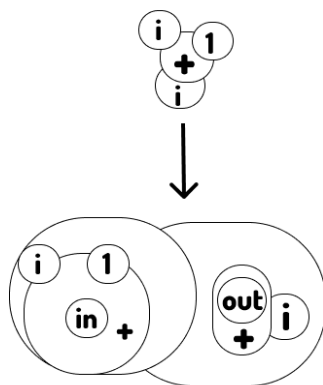
'Fill' is a function generalization of fill which automates this process:

The below shows a generalization for incrementing 'i' for a 'for loop,' and the precise underlying representation.

Equivalent Text Code:

```
(i = 0; i < n; i++) {
    body
}
```

Figure 37: 'Fill' Explanation

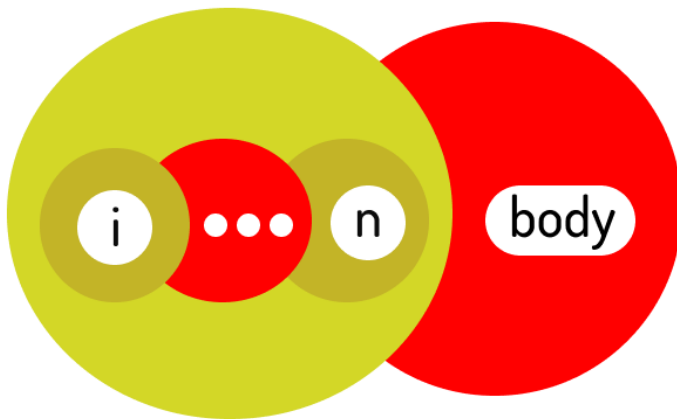


8.5.4 Bubble System ‘For Loop’ Symbolic Graphic

Figure 38 below is the graphical symbolic representation of a loop in the Bubble System Syntax. Figure 35 above shows a more technically exact version of this symbolic representation, used for the purpose of indirection from granular details.

Slots ‘i’ and ‘n’ are slots which are the parameters ‘for loop’. Once both ‘i’ and ‘n’ are filled, the body of the ‘for loop’ executes.

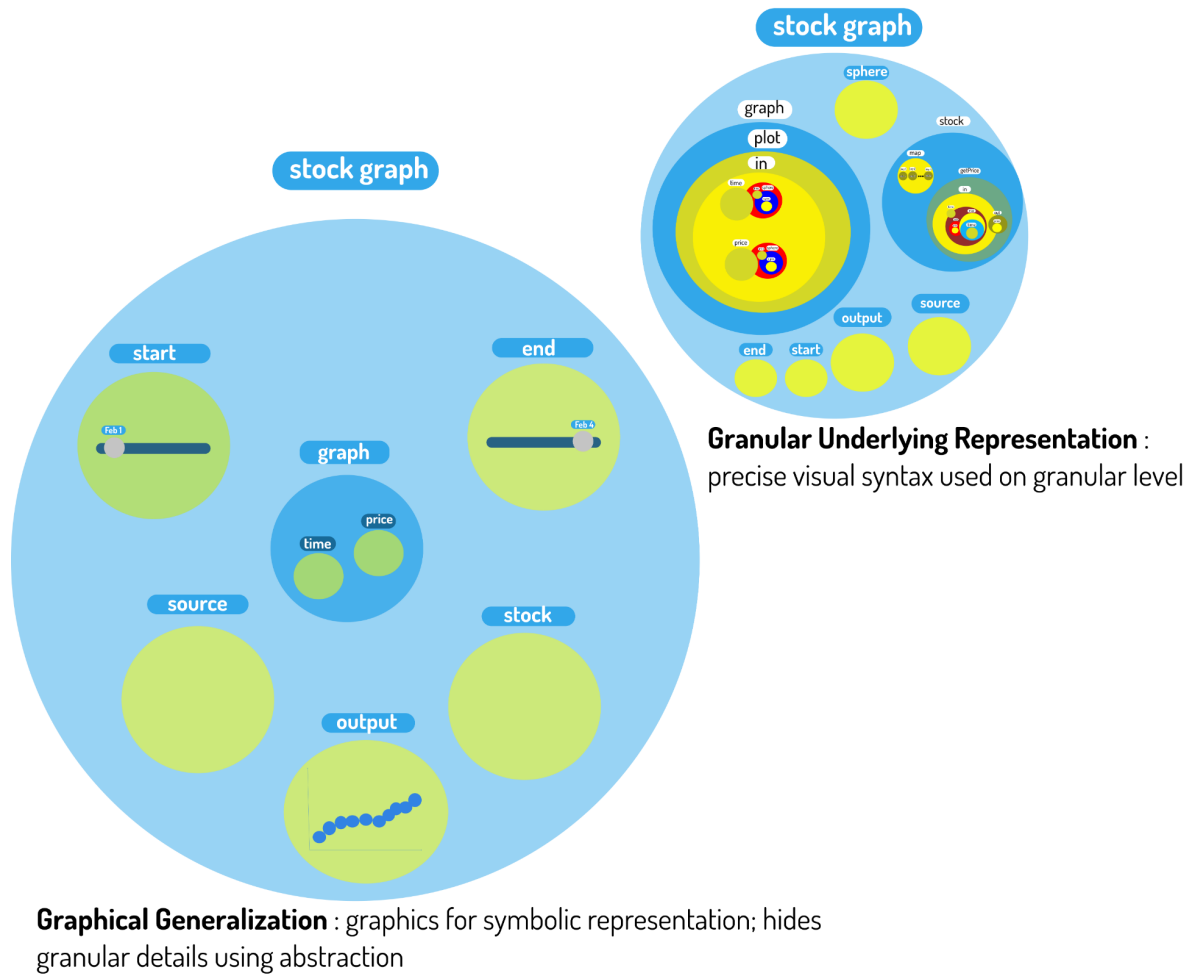
Figure 38: For Loop Symbolic Graphic



8.6 Generalization and Granularization

Below we draw the connections between the examples we have shown, outlining exactly what is a graphical generalization and what is a granular and precise underlying representation.

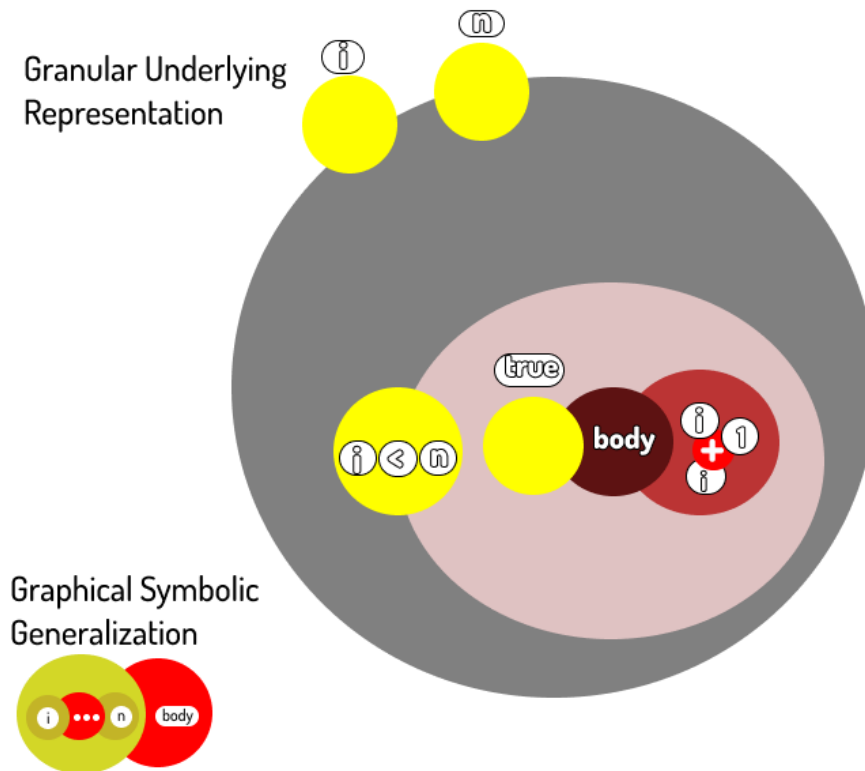
Figure 39: Stock Graph Generalization



8.6.1 Generalization and Granularization view for 'for loop'

See the diagrams below for an example of generalization and granularization being used. The symbolic generalization simply has the 'i' and 'n' acting like slots, but using a generalized graphic, hiding the granular details.

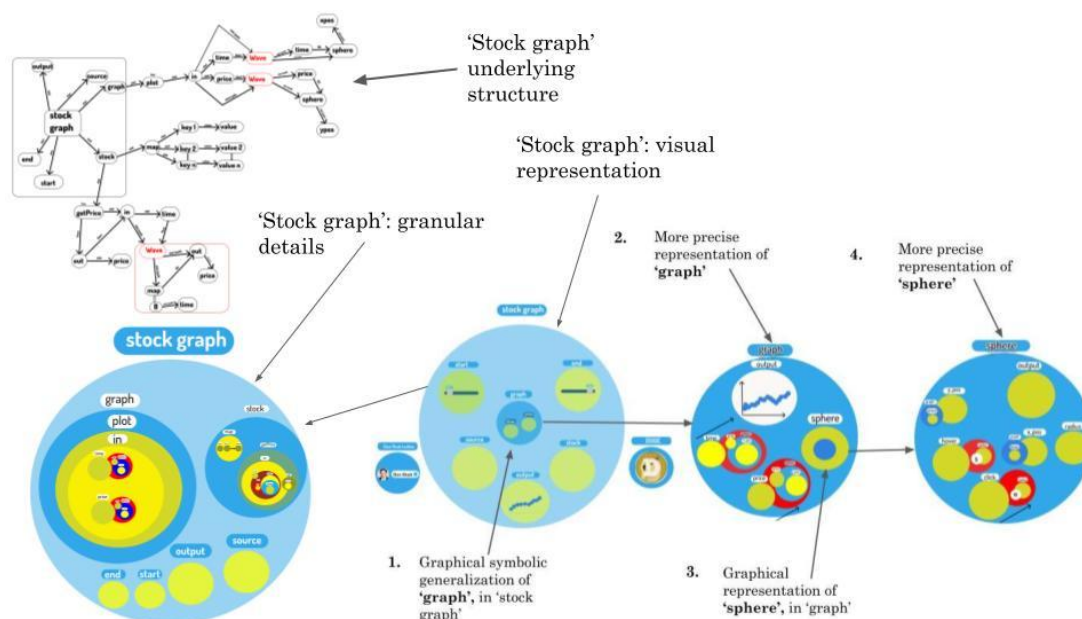
Figure 40: Generalization of 'For Loop'



8.6.2 The 'bigger' picture

Below in Figure 41 below is a diagram showing how the underlying representation connects with the graphical representations of code for the 'stock graph' example.

Figure 41: The 'bigger' picture



9. User Interface of VR Application built in the Bubble 3D Graphics System

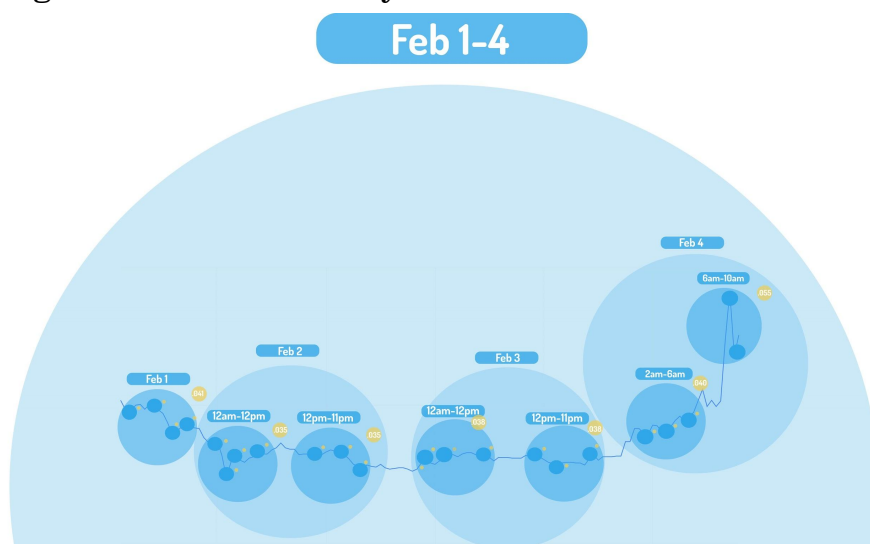
Figures 42-50 show the 3D output (user interface) and the features of the previously depicted VR data visualization (stock graph). This mockup of a data visualization application shows how Bubble makes it easy to create simple VR applications.

Note, this is a mockup, so the zooming will look a lot more immersive in a 3D VR environment. The Y axis of the graph represents price, and the X axis represents time. We see Elon Musk's tweets on the graph (in the 2 am circle on the left) as well along the graph of the price (the tweets are shown on the same line as the price of the stock). The blue dots change their Y position based on changes in price over time. The yellow dots show the actual price at the time for each circle in dollars.

9.1. Feb 1-4 Diagram zoomed-out

Figure 42, shown below, is our example of the stock graph from a zoomed out point of view. It shows the price of Dogecoin from Feb 1 - Feb 4, using the number slider for the slots "start" and "end" previously mentioned. Each bubble shown in the graph is a generalized set of points on the stock graph for a section of time. The days from 'Feb 2' to 'Feb 4' each have two bubbles for two parts of that day, which contain other bubbles inside. The user can zoom in and out of the graph, and click on a bubble to view it closer. In VR, this graph fully surrounds the user's field of view.

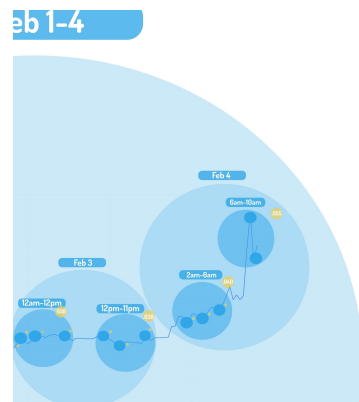
Figure 42: Feb 1-4--Fully zoomed out view



9.2. Feb 3-4 Diagram zoomed in

Figure 43, below, is zoomed in to Feb 3 and Feb 4, a closer view of Figure 42. This is another example of zooming in to make data bubbles more detailed.

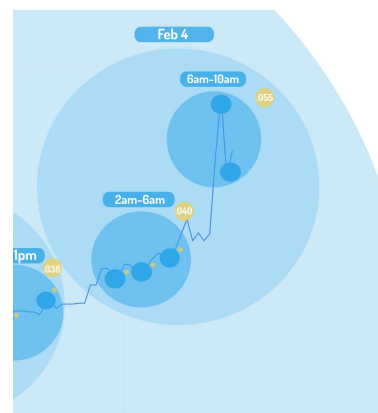
Figure 43 (29): Feb 3-4 zoomed in



9.3. Feb 4 Diagram

Figure 44, below, showing Feb 4, is a zoomed-in version of Figure 42. This is another example of using zooming in on the interface to change the generality of data being shown.

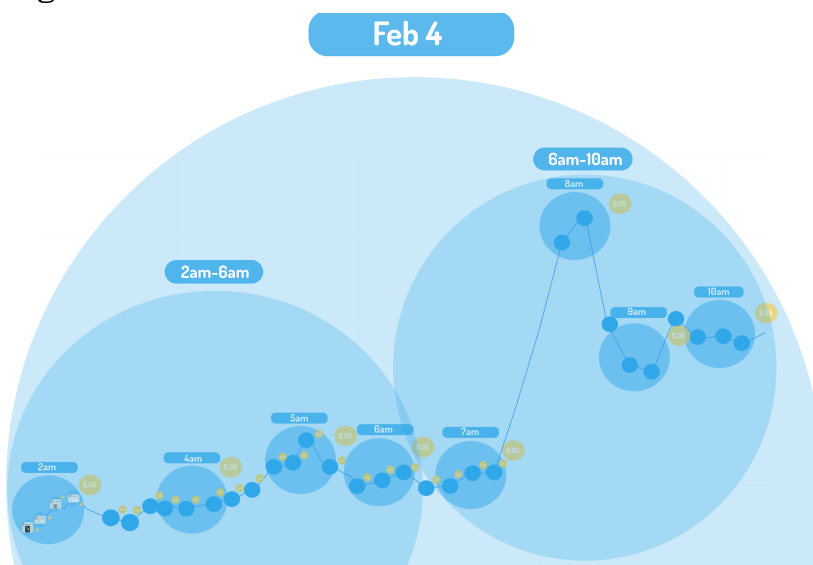
Figure 44: Feb 4:



9.4 Feb 4 Detailed

Figure 45, below, zoomed in on the Feb 4 data shown in Figures 44, viewing Feb 4 with greater detail. Notice how we were at a lower zoom level and less generality of interface in Figure 45 is the more detailed and zoomed in version of the same data. Notice that all of the tweets are at around 2 am. Due to the great price increase between the time of the tweets around 2 am, bottom left in Figure 45, also shown in Figure 46, showing that the tweets potentially influenced the price. When the tweets started (see on the circles on left at 2 am), the price level was far lower, and preceded a huge surge in price. We will see later as we zoom in the graph for the time when the tweets were made.

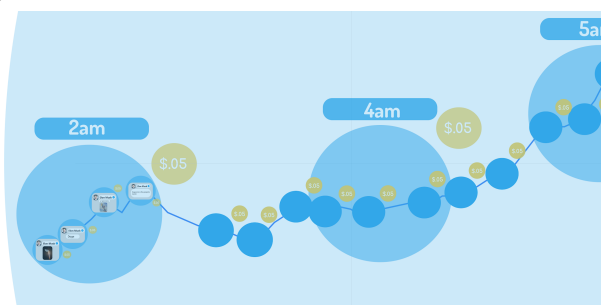
Figure 45: Feb 4 Detailed



9.5 Feb 4 2am-5am

Figure 46, shown below, is zoomed in on 2 am-5 am, is a zoomed in version of Figure 45. This is another example of zooming in to increase the density of the data bubbles shown on the interface. We see here Elon Musk's tweets on the graph along with the moving price. Note again, that all of the tweets are approximately at 2 am.

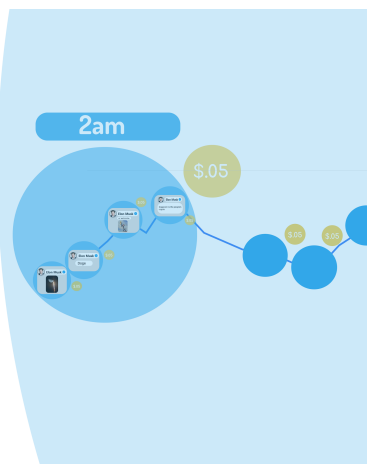
Figure 46: Feb 4 2am-5am



9.6 Feb 4 2am

See Figure 47, below, a zoomed in version of Figure 46, to gain a closer view of the data shown.

Figure 47: Feb 4 2am



9.7 2am Tweets

Figure 48 below is a zoomed in version of Figure 47.

Here we see each tweet up close on the stock graph line. The tweets are put on the same line as the price, so we can see the price when each tweet is made. As we will see later, zooming out will give us a view of how the tweets potentially influence the price. This particular tweet below is Elon Musk tweeting a rocket launch image, quickly followed by the word 'Doge.' Some analysts found this to be a tweet of support for Doge, and it preceded a rise in the price of the cryptocurrency later that morning.

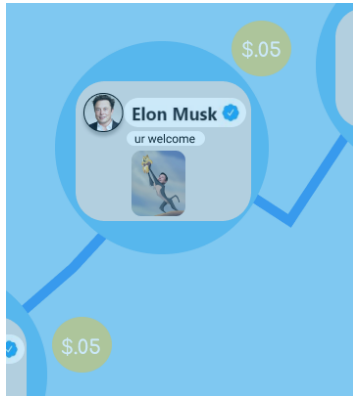
Figure 48: 2am tweets



9.8 Tweet: Detailed

Figure 49, below, is a zoomed in version of Figure 47 showing one detailed tweet.

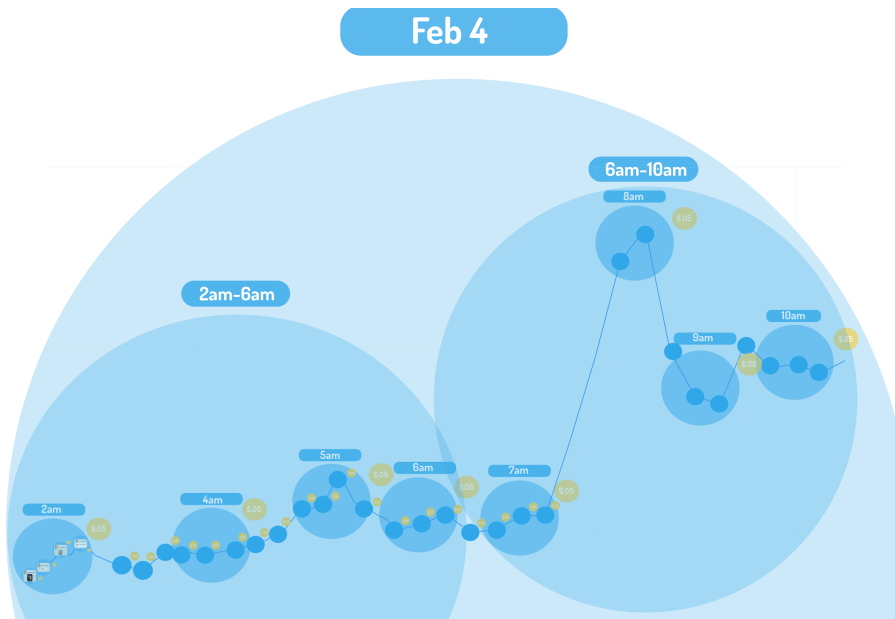
Figure 49: Tweet, Detailed:



9.9 Zoom Out to Feb 4

Figure 50 is a zoomed out view again just to put all of the data in context; notice the tweets from 2am on Feb 4 in the lower left corner. The previous figures shown were all zoomed in views of Feb 4.

Figure 50, below: Zoom out to Feb 4



10. Discussion

10.1 EDGE, CUBE and Bubble Comparison

There are some similarities between the relevant literature on visual programming and the Bubble System.

10.1.1 Knowledge Base

The Knowledge Base which stores Events in EDGE (see Chapter 3) is similar to the Bubble codebase [9]. EDGE has an EDG graph for visualizing multithreaded interactions, which is similar to Wave graphs in Bubble. In EDGE event relationships are stored in the knowledge base, similar to how the Bubble codebase can store Wave graphs, which are visual representations of the various conditions of state using a directed graph. Both Wave graphs and the EDGE graph are used for visualizing multithreaded interactions. Visualizing event timelines, events caused by actions, and viewing various levels of abstraction of the interface are additional important similarities between Bubble and EDGE [9]. EDGE visualization is done within a 2D interface. In the Bubble System, this visualization is done with a 3D interface for viewing and traversing abstraction levels using zoom to increase and decrease the generality being shown.

10.1.2 Dataflow and variable setting similarity

Within the CUBE system, the dataflow metaphor and ‘holder cubes’, which contain other cubes, (see Chapter 4) is a recursive and modular structure similar to the Bubble system, which also has a recursive structure, where wave graphs contain subgraphs [4]. The idea of putting a value into a cube to update the value is similar to putting a bubble into a slot in the Bubble System. In both cases, dragging a value into a variable updates the value of a variable.

10.1.3 Transition Networks and Waves Comparison

The Transition Network graphs described in the CUBE system (see Chapter 4) are similar to the Wave Graphs in Bubble, in that they both model state transitions [9]. Kleyn writes that each vertex in the graph can be a subdiagram, which is the case with wave graphs, where each wave can contain subgraphs, which are only shown when they are clicked on and visited. These further waves can be generalized. Having these recursive sub-diagrams is an important similarity between both systems, and the visualization capability derived from these recursive sub-diagrams specific to visual programming systems.

10.1.4 Context Free Grammars, Interactive Pushdown Automata and Wave Graphs

Kleyn mentions Context Free Grammars and Interactive Pushdown Automata [9] for visualizing the structure of program output and state changes (see Chapter 4). In the Bubble System, Wave Graphs are used to visualize how changes in the user input can change the state of the program, similar to pushdown automata. Both are graphs for visualizing various states of programs, and can be applied to visualizing the structure of interfaces. Wave graphs in the Bubble System might be a more interactive way to use and visualize pushdown automata, a three dimensional visualization allowing for more nodes and branches for visualization, and the potential of creating larger structures.

10.1.5 Window Occlusion Problem (EDGE)

In EDGE, Kleyn mentions that the programmer can select and change the value of a slot (variable) using a file menu [9]. Kleyn describes an occlusion problem where file menus and windows crowd the screen space and that a window will block the slot which is being updated. This occlusion problem is also mentioned by Najork as a reason that his system, CUBE is a 3D programming system rather than 2D [4].

10.1.6 Three Dimensional Solution

The occlusion problem described above is solved in Bubble because it is a 3D programming environment with no 2D windows, and the value of the slot being set is never obscured in the 3D environment. In three dimensions, all objects have depth as well, and the user will not easily select the wrong object.

Additionally, the Control Flow Graphs mentioned in EDGE (see Chapter 3) are two dimensional, and are used through window interfaces to control the structure of the program [9]. Wave Graphs in Bubble are the three dimensional version of this concept, where the screen space is also maximized to prevent the occlusion issue which Kleyn mentions [9].

10.2 Bubble Compared to Rumbaugh's State Trees

10.2.1 State Trees

Rumbaugh uses a directed graph, in 'State Trees' for visualizing the paths of interactions [10]. In 'State Trees,' just as 'local variable' scope is in the state which it is contained within, in the Bubble System, *slots* are within the scope of the Bubble

which they are inside, visually (a bubble with slots is analogous to an object with fields.) An action changes the value of variables, and also passes information to other actions (See Chapter 3). In Bubble, **slots** are similar to the **'local variables'** and **'events'** described by Rumbaugh, while **waves** in Bubble are similar to the **'actions'** which he describes [10].

Just as in the State Trees system, where 'actions' happen in response to 'events,' in Bubble, waves fire after slots are filled. Additionally, just as the State Tree represents a multithreaded graph of the different paths of the user actions in the interface, wave graphs in Bubble represent multithreaded control flow branches rather than a single thread at a time. In both systems, it is important that the code for user input is not visible, but rather hidden by a level of abstraction, to allow for visualization of the branches of the graph, and to allow for greater level of manipulation.

10.2.2 Bubble

In summary, Bubble simplifies the process described by Rumbaugh, with fewer moving parts in the system, and the ability to view three dimensional graphs of branched control flow using wave graphs. In *three dimensions*, control flow graphs can branch in all directions and not take up too much screen space, as mentioned by Najork [4]. Three dimensional space is a great way to use the power of visualizing a multithreaded graph, since the graph can be traversed through three dimensional zoom, in addition to moving throughout the graph in the same way that three dimensional objects can be moved throughout and manipulated.

State Trees are used to edit the interface of an application and wave graphs in Bubble have a similar function. In both systems, changing the graph changes the interface without rewriting code, a modular approach to building interfaces.

10.3 Discussion on Weaknesses of Visual Programming

Najork describes a few weaknesses of visual programming [4]. Here, we will outline how Bubble addresses these issues. Najork discusses possible disadvantages of visual languages:

- 1) Screen space.
- 2) Potential for slower runtime and greater overhead .
- 3) The fact that some visual languages lack static type systems.

He mentions that CUBE was the first to use higher order functions and dataflow in addition to using a visual representation to build code. Additionally, Najork writes that some visual languages can be inefficient because the code structure underlying the language runs on a structure too similar to the visual representation, rather than an optimization of the underlying code.

10.3.1 Bubble

Bubble avoids these above mentioned issues by optimizing code, which underlies the system instead of making the structure of the code solely based on the visual representation, which decreases the run-time compared to other visual languages. Using slots for building higher order functions efficiently reduces runtime and overhead and promotes code reuse making the implementation even more efficient. This is done by creating waves with multiple slot fills being done to create higher order functions which can be easily visualized.

10.3.2 Screen Space

Visual representations in Bubble can be “collapsed” using generalized graphics similar to the zoom feature in Google Maps, where zooming out makes the data being shown more general. This solves the problem of screen space because the user can change the amount of information shown by the interface, by changing the level of generalization. Najork argues that visual abstraction can fix the issue of graphs taking up a lot of screen space, describing this as using a symbol to “collapse sub-diagrams,” functioning as already built operators [4].

10.3.3 Types in Bubble

Najork cautions against visual languages being latently typed; Bubble is not latently typed, which reduces bugs and allows for more code reuse. The Bubble System cannot have type errors, because users cannot set a value of a slot to a bubble of the wrong type (types are checked before the code itself is even written). This means users can’t set a variable to an object of the incorrect type, minimizing the bugs which are inherent to latently typed languages.

10.3.4 Higher Order Functions

As Najork wrote, higher order functions, functions which return other functions, enable modularity. Visualizing abstractions is possible using a visual programming system because all of the abstraction levels of the different functions can be viewed. This makes for a more modular system, and one which enables visualizing its structure.

This is possible in Bubble because unlike most visual languages, Bubble is not first-order and allows for higher order functions to increase efficiency and code reuse. Najork mentions that many visual languages have issues because they are latently typed, and only check for type safety at runtime, which is a strong disadvantage for the overhead of many visual languages [4].

10.4. Future Direction - Type Graph for Visualizing Bubble Edits

Type Graphs can be created in the Bubble System to visualize all of the changes made to bubble objects. This Type Graph is in the form of a Wave Graph, which will be described in Chapter 7.6. The Type Graph, which is a directed graph, immediately shows all of the changes to bubbles; the slot changes to bubbles are immediately visible by showing in each wave the new slot values. The graphical changes to the bubble will be shown, as well. For example, we may start with a 'sphere' bubble as the initial type, with two types attached to it: a 'sphere' with another 'sphere' next to it, and a different type which is a bubble inside of a bubble. These type changes are shown visually through visual representations stored in each wave representing the change. This type graph is a future direction for the Bubble System, and will be useful for visualizing inheritance of many different bubble types. The Type Graph can be zoomed in and out of, traversed, and easily understood because of its visual nature. This visual Type Graph representation is similar to what Kleyn describes in EDGE; showing the changes to objects on a directed graph [9].

11. Summary and Conclusion

This paper outlines the concept of a visual programming system, Bubble, and illustrates the basic codebase prototype in addition to a mockup of an example application. In addition, This thesis outlines the history and current approaches of visual programming systems, introduces the concept of the bubble system as a potential new way for building VR applications, and discusses the innovations of the bubble system.

Bubble is not intended to replace text code. Some developers may just want to code using text. This is completely valid, and Bubble is not meant to erase the need for text programming. Bubble provides versatility for developers, who may want to build complex structures and use 3D graphics representations for visualization.

Bubble is designed to provide solutions for some of the issues outlined in the background section and innovations including:

- 1) An interface for building 3D graphics which is easier to use, since it is all one interface, and not multiple interfaces which are difficult to learn.
- 2) A three dimensional and Virtual Reality system for programming, similar to how Najork described in the CUBE system, which with the current increases in computing power is now more feasible to implement.
- 3) A codebase with pre-built code objects with visual representations, along with pre-built graphical objects, making it easier to reuse code and structures.
- 4) Wave Graphs: three-dimensional directed graphs for building abstract structures and logic, which builds on previous research done on directed graphs to represent programs.
- 5) Cross data visualization, so programmers can import datasets by simply searching the codebase and dragging a bubble into a slot

Due to time limitations, the Bubble System is not yet fully built and has not been tested. Therefore we don't really know how well this system will work, how much work is still needed to make it attractive for potential users, if users will adapt it, and if it is this groundbreaking new system that we hope it is. Despite these uncertainties, we plan to continue to work on finishing the system. Outstanding steps include:

- Fully building the codebase.
- Creating more visual representations of object oriented systems and dataflow, which we will store in our codebase for further use for fully building the system in addition to VR applications.
- Building interactive Wave graphs to visualize the dataflow while building complex VR applications. This makes the debugging process easier, since the programmer can visualize potential issues before they arise, simply by seeing the directed graph.
- Building prototype completely; user testing

The system requires a substantial amount of work and research to be completed. Ideally we should find funding for this work which will not be easy. It will be a challenging process to continue the research and project development without financial support. We know we are reaching high but we are optimistic that Bubble

will become what we envision. We are currently working on refining Bubble, making the graphics cleaner, and assembling the Wave Graphs (visual representations) of code for a chemistry VR application. This application will show features including: dragging elements together to create molecules, taking a molecule apart into separate elements, and zooming in and out of molecules at various levels of generalization.

12. List of Figures

Figure 1: Multiple Interfaces in Blender, Chapter 5.1

Figure 2: A ‘bubble,’ visual representation of an object, Chapter 7.2.3

Figure 3: A bubble named “Bubble” with a slot, Chapter 7.2.4

Figure 4: Close up of a slot, Chapter 7.2.4

Figure 5: Changing Slot Value, Chapter 7.2.5

Figure 6: Codebase Search, Chapter 7.3

Figure 7: Codebase Search Result, Chapter 7.3

Figure 8: Codebase drag search result to ‘Open Space’, Chapter 7.3

Figure 9: Wave Graph Example, Chapter 7.5.2

Figure 10: Slot in a Bubble, 7.6.1

Figure 11: Setting Value for Slots, 7.6.2

Figure 12: A Slot with a Wave, 7.6.3

Figure 13: A Bubble, with Slots, Chapter 7.7

Figure 14: The ‘Click’ Slot, Chapter 7.7.2

Figure 15: ‘Hover’ Slot, Chapter 7.7.3

Figure 16: Underlying Structure of the ‘Sphere’ Bubble

Figure 17: More Complex Example using Slots and Waves Syntax, ‘Graph’ Bubble, Chapter 7.8

Figure 18: Price Slot in ‘Graph’ Bubble, Chapter 7.8

Figure 19: Time Slot in 'Graph' Bubble, Chapter 7.8

Figure 20: 'Stock Graph' bubble with its slots, Chapter 8.2

Figure 21: Source Slot, Chapter 8.2

Figure 22: Start Slot, Chapter 8.2

Figure 23: End Slot, Chapter 8.2

Figure 24: Stock Slot, Chapter 8.2

Figure 25: Output Slot, Chapter 8.2

Figure 26: 'Graph' Slot, Chapter 8.2

Figure 27: 'Abstraction Levels,' Chapter 8.3

Figure 28: 'Stock Graph,' Highest Level of Abstraction Visual Code, Chapter 8.4

Figure 29: 'Graph' Bubble, Lower Level of Abstraction Visual Code, Chapter 8.4

Figure 30: 'Sphere' Bubble, Lowest Level of Abstraction Visual Code, Chapter 8.4

Figure 31: 'Stock Graph' Underlying Representation, Chapter 8.5

Figure 32: 'Stock Graph' Directed Graph, Chapter 8.6

Figure 33: 'Stock' Underlying Representation, Chapter 8.5.2

Figure 34: 'Graph' Underlying Representation, Chapter 8.5.2

Figure 35: 'For Loop' Syntax, Chapter 8.5.3

Figure 36: 'For Loop' Directed Graph, Chapter 8.5.3

Figure 37: ‘Fill’ Explanation, Chapter 8.5.3

Figure 38: ‘For Loop’ Symbolic Graphic, Chapter 8.5.4

Figure 39: ‘Stock Graph’ Generalization, Chapter 8.6

Figure 40: Generalization of ‘For Loop,’ Chapter 8.6.1

Figure 41: The ‘bigger’ picture, Chapter 8.6.2

Figure 42: Feb 1-4, Chapter 9.1

Figure 43: Feb 3-4 zoomed in, Chapter 9.2

Figure 44: Feb 4, Chapter 9.3

Figure 45: Feb 4 Detailed, Chapter 9.4

Figure 46: Feb 4 2am-5am, Chapter 9.5

Figure 47: Feb 4 2am, Chapter 9.6

Figure 48: 2am tweets, Chapter 9.7

Figure 49: Tweet, Detailed, Chapter 9.8

Figure 50: Zoom out to Original, Chapter 9.9

REFERENCES

- [1] Tomáš Mariančík, “Introducing LogiX - VR multiplayer visual programming system,” Channel: ‘Frooxius,’ 2 minutes, Feb 2017, https://www.youtube.com/watch?v=4rwYiINE9RE&ab_channel=Frooxius
- [2] Toni Härkönen, “Advantages and Implementation of Entity-Component-Systems,” Tampere University, Tampere, Finland, pp. 6, 13-15, 2019. <https://trepo.tuni.fi/bitstream/handle/123456789/27593/H%C3%A4rk%C3%B6nen.pdf?sequence=4&isAllowed=y>
- [3] Thibault Raffailac and Stéphane Huot, “Polyphony: Programming Interfaces and Interactions with the Entity-Component-System Model,” In: *Proceedings of the ACM on Human-Computer Interaction*, Vol. 3, EICS, Article 8, ACM, New York, NY. 22 pages. 2019. <https://doi.org/10.1145/3331150>
- [4] Marc Najork, “Programming in Three Dimensions,” In *Journal of Visual Languages and Computing*, Volume 7, Issue 2, pp. 219-242, 1996 <https://doi.org/10.1006/jvlc.1996.0012>
- [5] Ivan E. Sutherland, “Sketchpad: A Man-Machine Graphical Communication System” In *Proceedings of the SHARE design automation workshop (DAC '64)*. Association for Computing Machinery, New York, NY, USA, 1964, and University of Cambridge, Cambridge UK, 2003 <https://www.cl.cam.ac.uk/techreports/UCAM-CL-TR-574.pdf>
- [6] William R. Sutherland, “On-Line Graphical Specifications of Computer Procedures.” Ph.D. thesis, MIT, Cambridge, MA, 1966.
- [7] Alan Kay, "How Simply and Understandably Could The "Personal Computing Experience" Be Programmed?" *Intel Research Berkeley Programming Systems Seminar Series*, Nov 2006 https://www.youtube.com/watch?v=aqotNrIp-Ik&ab_channel=KennethFriedman
- [8] Alan Kay, “The early history of Smalltalk. ACM SIGPLAN Notices,” 28(3):69- 95, 1993. In: T.J. Bergin and R.G. Gibson, editors, *History of Programming Languages II*, pages 511-578. Addison-Wesley, 1996,
- [9] Michael Kleyn, Indranil Chakravarty “*EDGE* - A Graph-Based Tool for Specifying Interaction,” *Proceedings of the 1st Annual ACM SIGGRAPH*

Symposium on User Interface Software, Association for Computing Machinery, pp. 114, 1988.
<https://dl.acm.org/doi/pdf/10.1145/62402.62403>

[10] James Rumbaugh, "State Trees as Structured Finite State Machines for User Interfaces," *Proceedings of the 1st Annual ACM SIGGRAPH Symposium on User Interface Software*, Association for Computing Machinery, pp. 15-29, 1988.
<https://dl.acm.org/doi/pdf/10.1145/62402.62404>

[11] Ephraim P. Glinert, "Out of Flatland: Towards 3-D Visual Programming," In *Fall Joint Computer Conference*, pp 292-299, Dallas, TX, 1987.

[12] Colin Ware, Glenn Franck, "Viewing a graph in a virtual reality display is three times as good as a 2D diagram" In: *IEEE Symposium on Visual Languages*, St Louis, MO, IEEE Computer Society Press, Los Alamitos, CA, pp. 182-183. 1994

[13] Georg Raeder, "A Survey of Current Graphical Programming Techniques," In: *IEEE Computer Volume* University of Trondheim, Norway, 18, pp. 11-25, 1985.

[14] Randy Stiles and Michael Pontecorvo, "Lingua Graphica: A Visual Language for Virtual Environments" In: *IEEE Workshop on Visual Languages*, Seattle, WA, IEEE Computer Society Press, Los Alamitos, CA, pp. 225-227, 1987

[15] Timothy Hickey, "PHYSICS-BASED 3D GAME DESIGN AS A FIRST COURSE IN COMPUTING" *Journal of Circuits Systems and Computers*, June 2013.

[16] Harry Mairson, Functional Geometry and the "Traité de Lutherie," *J. Violin Soc. Am.: VSA Proceedings Vol. XXVI, No. 2*, pp. 191-192, Fall 2015